NumPy, Measured

Arrays, dtypes, views, broadcasting and the mistakes that give no error

8 chapters · 29,279 words

Every number is reproduced by code that ships with the book.

Contents

Chapters

Chapter 1 — The array 3

Chapter 2 — Dtypes, and the answers that change silently 13

Chapter 3 — Views and copies 23

Chapter 4 — Indexing 33

Chapter 5 — Broadcasting 44

Chapter 6 — Ufuncs and vectorization 56

Chapter 7 — Linear algebra and random numbers 71

Chapter 8 — Putting it together 84

Appendices

Appendix A — Described, not run 96

Appendix B — What went wrong while writing 99

Exercises

Exercises — Chapter 1 101

Exercises — Chapter 2 102

Exercises — Chapter 3 103

Exercises — Chapter 4 104

Exercises — Chapter 5 105

Exercises — Chapter 6 106

Exercises — Chapter 7 107

Exercises — Chapter 8 108

Worked solutions

Solutions — Chapter 1 109

Solutions — Chapter 2 111

Solutions — Chapter 3 113

Solutions — Chapter 4 115

Solutions — Chapter 5 117

Solutions — Chapter 6 119

Solutions — Chapter 7 121

Solutions — Chapter 8 123

Chapter 1 — The array

> **Build** — a function `describe(a)` that prints the memory layout of any array, and a measurement of how much faster an array sum is than a list sum.
>
> **Theory kept** — an array is one block of memory plus three numbers per axis: the shape, the dtype and the strides. Nothing else.
>
> **Source** — New. The behavior is from the NumPy documentation for `numpy.ndarray`. Every number is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch01_array`, which is `examples/ch01_array.py`.

A NumPy array is a block of memory and a description of how to read it. The block holds raw values, side by side, with no gaps. The description tells NumPy where one row ends and the next row starts. When you know these two facts, most of NumPy is no surprise.

This chapter shows the description, measures what it buys, and shows one case where it buys nothing. Each function that the chapter uses has its printed signature and a parameter table at the place where it first appears. The section How it works shows where each number in the description comes from.

The description: shape, dtype, strides

An array has three attributes that fix its layout.

- `shape` is a tuple with the length of each axis.
- `dtype` is the type of each value. It fixes the size of one value in bytes, which is `itemsize`.
- `strides` is a tuple with the number of bytes to jump to move one step along each axis.

Make a 3-by-4 array of 32-bit integers and print the three attributes.

```
import numpy as np

a = np.arange(12, dtype=np.int32).reshape(3, 4)
print(a)
print("shape   ", a.shape)
print("itemsize", a.itemsize)
print("strides ", a.strides)
print("nbytes  ", a.nbytes)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
shape    (3, 4)
itemsize 4
strides  (16, 4)
nbytes   48
```

Read the strides as a rule. To move one column to the right, jump 4 bytes, which is one `int32`. To move one row down, jump 16 bytes, which is 4 values of 4 bytes. The array has 12 values of 4 bytes each, so `nbytes` is 48.

The address of the value at row `i` and column `j` is therefore the start of the block plus `i * 16 + j * 4`. There is no other rule. Slicing, transposing and reshaping all work the same way. They change these numbers and do not touch the block.

The function `np.arange` made the values of this array. Its signature and parameters follow.

`np.arange`

```
np.arange(start_or_stop, /, stop=None, step=1, *, dtype=None, device=None, like=None)
```

The function returns an array of evenly spaced values. It has the same role as the Python `range`, and it accepts floats.

parameter	default	what it does	what goes wrong
`start_or_stop`	required	with one argument, the stop value and the start is 0. With two, the start	—
`stop`	`None`	the end of the range. The documented intent is that it is not included	for float steps the last value can be above it (next section)
`step`	`1`	the distance between values. A negative step counts down	a float step gives a count that depends on rounding
`dtype`	`None`	the type of each value. With `None`, NumPy takes an integer type if all arguments are integers, and `float64` if not	a range that is too large for the dtype wraps without an error: 300 values in `int8` end at 43
`device`, `like`	—	for use with other array libraries	—

The last row is a short form: both parameters have the default `None`, and you leave them unset.

Making arrays

Three more functions make most of the arrays that you need. Each gives the shape and the dtype that you ask for, and nothing else.

`np.zeros`

```
np.zeros(shape, dtype=None, order='C', *, device=None, like=None)
```

The function returns a new array of the given shape, filled with zeros. Its relatives `np.ones`, `np.full` and `np.empty` take the same first arguments. The function `np.empty` does not set the values, so the block holds whatever was in that memory before. Never read an array from `np.empty` before you write to it.

parameter	default	what it does	what goes wrong
`shape`	required	an integer or a tuple of integers. The value `3` gives 3 values. The value `(2, 3)` gives 2 rows of 3	a bare `2, 3` is an error: the second number is taken as the dtype
`dtype`	`None`	the type of each value. With `None`, the type is `float64`	the default is a float, not an integer
`order`	`'C'`	`'C'` puts the last axis in consecutive memory, `'F'` puts the first axis there	—

`np.linspace`

```
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0, *,
device=None)
```

The function returns `num` values that are evenly spaced between `start` and `stop`. Use it when you know how many values you want. Use `np.arange` when you know the step.

parameter	default	what it does	what goes wrong
`start`, `stop`	required	the first value and the last value of the range	—
`num`	`50`	the number of values	—
`endpoint`	`True`	with `False`, the range does not include `stop`, and the step is `(stop - start) / num`	the step changes when you switch it
`retstep`	`False`	with `True`, the function returns the pair `(array, step)`	the result is a tuple, not an array
`dtype`	`None`	the type of the result. With `None`, a float	an integer dtype truncates the values

The two functions differ in one important way. The step of `np.arange` is given, and the count is computed from floats. The count of `np.linspace` is given, and the step is computed.

```
import numpy as np

r = np.arange(1, 1.3, 0.1)
print("arange(1, 1.3, 0.1) has", len(r), "values")
print("last value:", r[-1], "and the stop was 1.3")
print("last value is above the stop:", r[-1] > 1.3)
print("quotient (stop - start) / step:", (1.3 - 1) / 0.1)

s = np.linspace(1, 1.3, 4)
print("linspace(1, 1.3, 4):", s)
```

```
arange(1, 1.3, 0.1) has 4 values
last value: 1.3000000000000003 and the stop was 1.3
last value is above the stop: True
quotient (stop - start) / step: 3.0000000000000004
linspace(1, 1.3, 4): [1.  1.1 1.2 1.3]
```

The range `1` to `1.3` with step `0.1` has the quotient `(1.3 - 1) / 0.1`. In floats this is 3.0000000000000004, and NumPy rounds up. It makes 4 values, and the last one is larger than the stop. When you need a known number of values,

use `np.linspace`.

`np.array`

```
np.array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, ndmax=0,
like=None)
```

The function builds an array from Python objects. It is the usual way to enter data by hand.

parameter	default	what it does	what goes wrong
`object`	required	a list, a tuple, a number or another array. Equal-length nested lists make a 2-D array	unequal rows are an error in NumPy 2
`dtype`	`None`	the type of each value. With `None`, NumPy takes the smallest type that holds all the values	a mix of numbers and text becomes text
`copy`	`True`	`True` always copies. `None` copies only when needed. `False` raises an error if a copy cannot be avoided	`True` costs memory for a large input
`order`	`'K'`	the memory order of the result. `'K'` keeps the order of the input	—
`ndmin`	`0`	the least number of axes. NumPy puts length-1 axes on the left to reach it	—

The function picks the dtype for you, and sometimes the choice is not the one that you expect.

```
import numpy as np

print(np.array([1, 2, 3]).dtype)
print(np.array([True, 2]).dtype)
print(np.array([1, 2.5]).dtype)
print(np.array([1, "a"]).dtype)

try:
    np.array([[1, 2], [3]])
except ValueError as e:
    print("ValueError:", str(e).split(".")[0])
```

```
int64
int64
float64
<U21
ValueError: setting an array element with a sequence
```

A mix of an integer and a string becomes an array of strings, `<U21`, and the number 1 becomes the text `'1'`. A list of unequal rows is an error in NumPy 2. NumPy 1.x made an array of objects for the same call. Check the dtype of any array that you build from mixed data.

A transpose copies nothing

Transpose the array and print its layout again.

```
import numpy as np

a = np.arange(12, dtype=np.int32).reshape(3, 4)
b = a.T
print("a strides", a.strides, "shape", a.shape)
print("b strides", b.strides, "shape", b.shape)
print("same memory:", np.shares_memory(a, b))
print("b is C-contiguous:", b.flags.c_contiguous)
```

```
a strides (16, 4) shape (3, 4)
b strides (4, 16) shape (4, 3)
same memory: True
b is C-contiguous: False
```

The shape went from `(3, 4)` to `(4, 3)`, and the strides swapped from `(16, 4)` to `(4, 16)`. The 48 bytes did not move. The function `np.shares_memory` confirms that `a` and `b` read the same block.

`np.shares_memory`

```
np.shares_memory(a, b, /, max_work=-1)
```

The function returns `True` if the two arrays have at least one byte of memory in common.

parameter	default	what it does	what goes wrong
`a`, `b`	required	the two arrays to compare	—
`max_work`	`-1`	how much effort to spend. `-1` solves the problem exactly. `0` compares only the first and last byte of each array	the exact answer can take very long for complex strides

The last line of the block above shows the price of a transpose. A C-contiguous array has its last axis in consecutive memory, so a walk along the last axis reads memory in order. The transpose does not have this property. A loop over its last axis jumps 16 bytes at each step. Chapter 3 measures what that costs.

The function you build

Put the three attributes and the two flags that matter into one function. You will use it in every later chapter.

```
import numpy as np


def describe(a: np.ndarray) -> str:
    """One line that tells the whole layout of an array."""
    return (f"shape={a.shape} dtype={a.dtype} itemsize={a.itemsize} "
            f"strides={a.strides} nbytes={a.nbytes} "
            f"c_contiguous={a.flags.c_contiguous} owns={a.flags.owndata}")


a = np.arange(12, dtype=np.int32).reshape(3, 4)
print(describe(a))
print(describe(a.T))
print(describe(a[:, ::2]))
```

```
shape=(3, 4) dtype=int32 itemsize=4 strides=(16, 4) nbytes=48 c_contiguous=True owns=False
shape=(4, 3) dtype=int32 itemsize=4 strides=(4, 16) nbytes=48 c_contiguous=False owns=False
shape=(3, 2) dtype=int32 itemsize=4 strides=(16, 8) nbytes=24 c_contiguous=False owns=False
```

`describe`

```
describe(a: numpy.ndarray) -> str
```

The function reads attributes only, and it never copies the array.

parameter	default	what it does	what goes wrong
`a`	required	any `ndarray`, of any shape and dtype	a Python list has no `.shape`, so the call raises `AttributeError`

The `owns` field is `True` when the array owns its block. It is `False` when the array only looks at the block of another array. The slice `a[:, ::2]` takes every second column. It has strides `(16, 8)`, so it reads the same block with a step of 8 bytes.

The first line is a surprise. The array `a` itself has `owns=False`. The call `np.arange(12)` made the block, and `reshape` returned a new view of that block. The variable `a` is a view, and the block belongs to an array that no name points to. Chapter 3 shows why this matters.

Why a sum over an array is fast

A Python list holds pointers. Each pointer leads to a separate integer object somewhere in memory. A sum over the list must follow each pointer, check the type of the object, and add. An array holds the values themselves, so a sum is one tight loop over consecutive bytes.

(This describes the CPython object model. The blocks below measure the time and the memory that follow from it. They do not inspect the interpreter. Appendix A lists this statement.)

Measure both. The code below sums one million integers in each form and keeps the best of five runs.

```
import timeit

import numpy as np

n = 1_000_000
as_list = list(range(n))
as_array = np.arange(n)

t_list = min(timeit.repeat(lambda: sum(as_list), number=5, repeat=5)) / 5
t_array = min(timeit.repeat(lambda: as_array.sum(), number=5, repeat=5)) / 5
ratio = t_list / t_array
assert sum(as_list) == int(as_array.sum())
assert ratio > 5, "chapter 1 claims an array sum is at least 5 times faster"
print(f"list sum   {t_list * 1e3:8.3f} ms")
print(f"array sum  {t_array * 1e3:8.3f} ms")
print(f"ratio      {ratio:8.1f}x")
```

```
# output varies
list sum      9.604 ms
array sum     0.287 ms
ratio          33.4x
```

The numbers above are from one run on a shared machine. Yours will differ. The `assert` is the claim: the array sum is at least 5 times faster. The assert fails if the ratio falls below 5 on your machine.

Memory shows the same effect. The list holds 8-byte pointers and, in addition, one 28-byte object for each integer above 256.

```
import sys

import numpy as np

n = 1_000_000
as_list = list(range(n))
list_bytes = sys.getsizeof(as_list) + sum(sys.getsizeof(x) for x in as_list)
array_bytes = np.arange(n).nbytes
print("list  ", list_bytes, "bytes")
print("array ", array_bytes, "bytes")
print("ratio ", round(list_bytes / array_bytes, 1))
```

```
list   36000056 bytes
array  8000000 bytes
ratio  4.5
```

The list needs 4.5 times the memory of the array for the same one million integers. The ratio depends on the dtype. An `int8` array of the same length is 36 times smaller than the list.

A case where the array is slower

The array has a fixed cost for each call. NumPy must check the type, allocate a result and start the loop. For a very small input, this cost is larger than the work.

```
import timeit

import numpy as np

small_list = [1, 2, 3]
small_array = np.array(small_list)
t_list = min(timeit.repeat(lambda: sum(small_list), number=20000, repeat=5)) / 20000
t_array = min(timeit.repeat(lambda: small_array.sum(), number=20000, repeat=5)) / 20000
print("array is slower for 3 values:", t_array > t_list)
assert t_array > t_list
```

```
array is slower for 3 values: True
```

The list wins for three values. Use NumPy when you have thousands of values or more, or when the shape of the problem is a table. For a handful of numbers, use a list and do not apologize.

How it works

Everything above follows from one fact: the array is a block of bytes and three attributes that say how to read it. This section opens the block and checks each claim against the bytes.

An array of four 16-bit integers is eight bytes in a row.

```
import numpy as np

a = np.arange(4, dtype=np.int16)
print(a.tobytes())
print(list(a.tobytes()))
```

```
b'\x00\x00\x01\x00\x02\x00\x03\x00'
[0, 0, 1, 0, 2, 0, 3, 0]
```

Each integer uses two bytes, and the low byte comes first. This is the little-endian order of most computers. The block has no header, no length and no type. The array object holds those facts separately, and `strides` is how it finds a value in the block.

The rule for the address is general. For the index `(i0, i1, ..., ik)`, the byte offset is the sum of `i_n * strides[n]`. The next block tests this rule on a 3-axis array. It reads every value from the raw bytes with the formula, and compares it with the value that NumPy returns.

```
import itertools

import numpy as np

a = np.arange(24, dtype=np.int16).reshape(2, 3, 4)
raw = a.tobytes()


def read_at(index):
    offset = sum(i * s for i, s in zip(index, a.strides))
    return int.from_bytes(raw[offset:offset + a.itemsize], "little", signed=True)


every = list(itertools.product(*[range(n) for n in a.shape]))
print("strides:", a.strides)
print("values checked:", len(every))
print("all equal:", all(read_at(ix) == a[ix] for ix in every))
```

```
strides: (24, 8, 2)
values checked: 24
all equal: True
```

The strides come from the shape. The last axis has stride 2, which is the item size. Each earlier stride is the next stride times the next length: 2 × 4 = 8, and 8 × 3 = 24. This is C order, also called row-major order. In C order, the last axis changes fastest in memory.

The other common order is Fortran order, or column-major order. The first axis changes fastest. NumPy can hold the same values in either order, and only the strides differ.

```
import numpy as np

c = np.arange(24, dtype=np.int16).reshape(2, 3, 4)
f = np.asfortranarray(c)

print("C strides:", c.strides, "C-contiguous:", c.flags.c_contiguous)
print("F strides:", f.strides, "F-contiguous:", f.flags.f_contiguous)
print("same values:", np.array_equal(c, f))
print("same memory:", np.shares_memory(c, f))
```

```
C strides: (24, 8, 2) C-contiguous: True
F strides: (2, 4, 12) F-contiguous: True
same values: True
same memory: False
```

The Fortran strides `(2, 4, 12)` run the other way: 2, then 2 × 2 = 4, then 4 × 3 = 12. The two arrays hold equal values in two blocks, so `asfortranarray` made a copy.

The invariant. The byte offset of an element is always the sum of index times stride, for any shape, any order and any view. A test of this is the block above.

The failure. Code can assume a C-ordered block and receive another order. It then reads the right values slowly. Or it passes a wrong pointer to a library that expects one order. Chapter 3 measures the slow case. The default order is C.

Reference

Callable	Introduced in	Contract
`np.arange`	The description: shape, dtype, strides	evenly spaced values, with a given step
`np.zeros`	Making arrays	an array of zeros of a given shape and dtype
`np.linspace`	Making arrays	a given count of evenly spaced values
`np.array`	Making arrays	an array from Python objects
`np.shares_memory`	A transpose copies nothing	exact test for a shared byte
`describe`	The function you build	one line with the layout of an array

What you have now

You can read any array as a block plus `(shape, dtype, strides)`, and you know where each stride comes from. You can make arrays with four functions, and you know what their parameters do and where each one goes wrong. You have a `describe` function that prints the layout.

You also measured three results. The array sum is at least 5 times faster, which the assert proves. The memory is 4.5 times smaller. Three values are faster in a list.

Chapter 2 looks at the `dtype` field, which decides what a value is, and at the ways it silently changes the answer.

Chapter 2 — Dtypes, and the answers that change silently

> **Build** — `checked_add(a, b)`, an integer add that raises an error when a result does not fit the dtype, in place of wrapping around.
>
> **Theory kept** — a dtype fixes how many bits a value has. NumPy does not check that a result fits in those bits. It keeps the bits that fit and drops the rest.
>
> **Source** — New. The promotion rules are NEP 50, which NumPy 2 implements. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch02_dtypes`, which is `examples/ch02_dtypes.py`.

Chapter 1 gave every array a dtype. This chapter shows what the dtype does to your numbers. Four things go wrong in practice.

- Integers wrap.
- Floats lose small additions.
- A sum uses a different dtype than its input.
- A mixed expression picks a dtype that you did not choose.

Each one gives a wrong answer and no error.

What NumPy picks when you do not choose

A Python integer becomes `int64` on this machine. A Python float becomes `float64`. A mix of the two becomes `float64`.

```
import numpy as np

print(np.array([1, 2, 3]).dtype)
print(np.array([1.0, 2]).dtype)
print(np.array([1, 2.5]).dtype)
print(np.array([1, 2, 3]).sum().dtype)
print((np.array([1, 2, 3]) / 2).dtype)
```

```
int64
float64
float64
int64
float64
```

The last line is a common surprise. The division operator `/` always gives floats, even for integer input. Use `//` for integer division.

Integers wrap around

An `int8` holds the values −128 to 127. NumPy does not stop you at the edge. It wraps.

```
import numpy as np

a = np.array([100, 120, 127], dtype=np.int8)
print(a + 1)
print((a + 1).dtype)

img = np.array([200, 250, 10], dtype=np.uint8)
print(img + 100)
print(np.array([300, -5]).astype(np.uint8))
```

```
[ 101  121 -128]
int8
[ 44  94 110]
[ 44 251]
```

The value 127 plus 1 is −128. The pixel 200 plus 100 is 44, not 300 and not 255. The call `astype(np.uint8)` turned 300 into 44 and −5 into 251. NumPy printed no message for any of these.

`np.ndarray.astype`

```
np.ndarray.astype(self, /, dtype, order='K', casting='unsafe', subok=True, copy=True)
```

The method returns a copy of the array with another dtype. It is the tool that changed 300 into 44 above.

parameter	default	what it does	what goes wrong
`dtype`	required	the new type	a narrower integer type wraps the values: 300 becomes 44
`order`	`'K'`	the memory order of the result. `'K'` keeps the order of the input	—
`casting`	`'unsafe'`	which casts are allowed. `'safe'` allows only casts that cannot lose a value	with `'unsafe'` nothing stops 300 from becoming 44. With `'safe'` the call raises `TypeError`
`copy`	`True`	with `False`, the method returns the same array when the dtype already matches	with `True` the call always makes a new block

The `casting` parameter is the guard. The call `np.array([300]).astype(np.uint8, casting='safe')` raises `TypeError: Cannot cast array data from dtype('int64') to dtype('uint8') according to the rule 'safe'`.

This is the failure that matters most for image data. A brightness boost on a `uint8` image does not clip at 255. It wraps, and the bright areas turn dark.

NumPy 2 does raise one error. A Python integer that does not fit the dtype of the array is an `OverflowError`, as the next block shows.

```
import numpy as np

img = np.array([200, 250, 10], dtype=np.uint8)
try:
    img + 300
except OverflowError as e:
    print(type(e).__name__, "-", e)
```

```
OverflowError - Python integer 300 out of bounds for uint8
```

The error is only for a Python integer that is too large for the dtype. It does not apply when both operands are arrays. The first example, `img + 100`, passed because 100 fits in `uint8`. The result of the add did not fit, and nobody checked.

The function you build

The rule for a safe add is short. Do the add in a wider type. Compare each result with the limits of the narrow type. Raise an error if one result is outside the limits. `np.iinfo` gives the limits.

```
import numpy as np


def checked_add(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Add two integer arrays. Raise OverflowError if a result does not fit the dtype of
`a`."""
    if a.dtype.kind not in "iu" or b.dtype.kind not in "iu":
        raise TypeError("checked_add takes integer arrays only")
    if a.dtype == np.uint64 or b.dtype == np.uint64:
        raise TypeError("uint64 has no wider integer type to check against")
    wide = a.astype(np.int64) + b.astype(np.int64)
    info = np.iinfo(a.dtype)
    if wide.size and (wide.min() < info.min or wide.max() > info.max):
        bad = int(np.flatnonzero((wide < info.min) | (wide > info.max))[0])
        raise OverflowError(f"element {bad}: {int(wide.flat[bad])} does not fit {a.dtype}")
    return wide.astype(a.dtype)


pixels = np.array([200, 250, 10], dtype=np.uint8)
try:
    checked_add(pixels, np.array([100, 100, 100], dtype=np.uint8))
except OverflowError as e:
    print("refused:", e)
print(checked_add(pixels, np.array([10, 5, 100], dtype=np.uint8)))
```

```
refused: element 0: 300 does not fit uint8
[210 255 110]
```

The function uses `np.iinfo` for the limits of the narrow type.

`np.iinfo`

```
np.iinfo(int_type)
```

The class returns the limits of an integer type. The object has the attributes `min`, `max` and `bits`.

parameter	default	what it does	what goes wrong
`int_type`	required	an integer dtype, a dtype name or an integer array	a float dtype raises `ValueError`. Use `np.finfo` for floats

`checked_add`

```
checked_add(a: numpy.ndarray, b: numpy.ndarray) -> numpy.ndarray
```

The function adds two integer arrays and raises `OverflowError` for the first result that does not fit the dtype of `a`.

parameter	default	what it does	what goes wrong
`a`	required	an integer array. Its dtype is the dtype of the result	a `uint64` array raises `TypeError`, because no wider type exists to check against
`b`	required	an integer array that broadcasts against `a`	a float array raises `TypeError`

The function has two limits, and the code says both. It cannot check `uint64`, because no wider integer type exists in NumPy. It takes `a.dtype` as the output type, so a `uint8` plus an `int16` is checked against `uint8`. For a clip instead of an error, use `np.clip(wide, info.min, info.max)`.

A sum has its own dtype

A sum over an `int8` array does not use `int8`. NumPy sums small integers in the platform integer, which is `int64` here. You can force the narrow type with `dtype=`, and the sum wraps.

```
import numpy as np

a = np.array([100, 120, 127], dtype=np.int8)
print(a.sum(), a.sum().dtype)
print(a.sum(dtype=np.int8))
print(a.mean(), a.mean().dtype)
```

```
347 int64
91
115.66666666666667 float64
```

The true sum is 347. The default sum gives 347. The forced `int8` sum gives 91, which is 347 minus 256. The mean is a `float64` and is correct.

`np.sum`

```
np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)
```

The function adds the values of an array along an axis. The method `a.sum(...)` takes the same arguments after `self`.

parameter	default	what it does	what goes wrong
`a`	required	the array to add	—
`axis`	`None`	the axis to add along. `None` adds all values. A tuple adds along several axes	—
`dtype`	`None`	the type of the running total. With `None`, an integer input smaller than the platform integer uses `int64`	`dtype=np.int8` on the values above gives 91 and not 347, with no warning
`keepdims`	`<no value>`	with `True`, the reduced axes stay with length 1	the shape of the result changes, and broadcasting differs

This default is a protection and also a trap. The sum of a `uint8` image is safe. The sum of a `float32` array stays `float32`, and the next section shows that this is not safe.

Floats lose small additions

A `float32` has 24 bits of precision. At 16,777,216, which is 2 to the power 24, the gap between two neighbor values is 2. The sum with 1 is the same number.

```
import numpy as np

print(np.float32(16777216) + np.float32(1))
print(np.float64(16777216) + 1)
print("float32 epsilon:", np.finfo(np.float32).eps)
print("float64 epsilon:", np.finfo(np.float64).eps)
```

```
1.6777216e+07
16777217.0
float32 epsilon: 1.1920929e-07
float64 epsilon: 2.220446049250313e-16
```

The `eps` value is the gap between 1.0 and the next float. For `float32` it is about 1.2e-7, so you get about 7 correct decimal digits. For `float64` you get about 15 or 16.

Now the practical case. Add ten million copies of 0.1 in `float32`. The exact answer is 1,000,000.015, because `float32(0.1)` is a little more than 0.1. Compare three ways to add.

```
import numpy as np

x = np.full(10_000_000, 0.1, dtype=np.float32)
exact = float(np.float32(0.1)) * x.size

pairwise = float(x.sum())
running = float(np.cumsum(x)[-1])
wide = float(x.sum(dtype=np.float64))
for name, value in [("x.sum()", pairwise), ("cumsum[-1]", running), ("sum float64", wide)]:
    print(f"{name:12s} {value:14.3f}  relative error {abs(value - exact) / exact:.2e}")
```

```
x.sum()         1000000.125  relative error 1.10e-07
cumsum[-1]      1087937.000  relative error 8.79e-02
sum float64     1000000.015  relative error 0.00e+00
```

The three results differ a lot. `x.sum()` has a relative error of 1e-7, which is the best a `float32` can give. The cumulative sum from `cumsum` is wrong by almost 9 percent, and it is too large, not too small. A cumulative sum adds a small number to a large total. The section How it works measures what each add does.

The NumPy documentation says that `x.sum()` adds in pairs. It adds neighbors first, then adds the partial sums, so the numbers that it adds stay close in size. This book did not test that explanation. The result shows that the pairwise sum is accurate. The function `cumsum` must produce every cumulative total in order, so it cannot add in pairs.

A pure Python loop has the same fault as `cumsum`. Chapter 6 uses this fact when it builds a moving average.

Two more functions show the limits of a `float32` and the cure for the running sum.

`np.finfo`

```
np.finfo(dtype)
```

The class returns the limits of a float type. The attributes `eps`, `max`, `tiny` and `bits` describe it.

parameter	default	what it does	what goes wrong
`dtype`	required	a float dtype, a dtype name or a float array	an integer dtype raises `ValueError`. Use `np.iinfo` for integers

`np.cumsum`

```
np.cumsum(a, axis=None, dtype=None, out=None)
```

The function returns the running total of an array. Every partial sum is in the result.

parameter	default	what it does	what goes wrong
`a`	required	the array to add	—
`axis`	`None`	the axis to add along. With `None`, the array is flattened first	a 2-D input becomes a 1-D result
`dtype`	`None`	the type of the running total and of the result	for ten million `float32` values of 0.1, the default is wrong by 8.8 percent, and `np.float64` gives 0.0 percent

Mixed dtypes: which one wins

When two dtypes meet, NumPy picks the smallest dtype that holds both. NumPy 2 added one rule for Python numbers. A Python `int` or `float` does not change the dtype of an array. It takes the dtype of the array, if it fits.

```
import numpy as np

print((np.float32(3) + 3.0).dtype)
print((np.float32(3) + np.float64(3)).dtype)
print((np.int8(1) + np.int16(1)).dtype)
print((np.int64(1) + np.uint64(1)).dtype)
print((np.array([1, 2], dtype=np.uint8) + 1).dtype)
```

```
float32
float64
int16
float64
uint8
```

Read the lines in order. A `float32` plus the Python float 3.0 stays `float32`. A `float32` plus a `float64` becomes `float64`. An `int64` plus a `uint64` has no integer type that holds both, so it becomes `float64`, and large integers lose exactness. The last line is the NumPy 2 rule: a `uint8` array plus the Python integer 1 stays `uint8`.

The documentation says that NumPy 1.x chose the dtype from the value of a Python number. This book did not install NumPy 1.x, so code that you find online can describe results that this book did not check. Check the version of NumPy before you trust old advice.

`np.can_cast`

```
np.can_cast(from_, to, casting='safe')
```

The function tells you whether a cast from one dtype to another is allowed under a rule.

parameter	default	what it does	what goes wrong
`from_`	required	the source dtype, or an array	—
`to`	required	the target dtype	—
`casting`	`'safe'`	the rule: `'no'`, `'equiv'`, `'safe'`, `'same_kind'` or `'unsafe'`	`'safe'` allows `int64` to `float64`, which can lose the last digits

The last row is worth a test. The call `np.can_cast(np.int64, np.float64)` returns `True`. A `float64` has 53 bits of precision, and an `int64` has 63. The next block shows the loss.

```
import numpy as np

print("can_cast int64 -> float64:", np.can_cast(np.int64, np.float64))
big = np.int64(2 ** 53 + 1)
print("int64 value:   ", int(big))
print("as float64:    ", int(np.float64(big)))
print("lost:          ", int(big) - int(np.float64(big)))
```

```
can_cast int64 -> float64: True
int64 value:    9007199254740993
as float64:     9007199254740992
lost:           1
```

The word "safe" means that the type has no overflow. It does not mean that every value stays exact.

How it works

Two facts explain every surprise in this chapter. An integer dtype does arithmetic modulo a power of 2. A float dtype stores a sign, an exponent and a fixed number of fraction bits.

Integers. An `int8` has 8 bits, and the add of two `int8` values keeps the low 8 bits of the result. This is arithmetic modulo 256. The bits are read as a signed number, in two's complement form. The next block shows that the bits do not change. It adds every pair of `int8` values, 65,536 pairs, and compares the result with arithmetic modulo 256.

```
import numpy as np

v = np.arange(-128, 128, dtype=np.int8)
x, y = np.meshgrid(v, v)
with np.errstate(over="ignore"):
    got = x + y
want = ((x.astype(np.int64) + y + 128) % 256) - 128

print("pairs checked:", got.size)
print("result dtype: ", got.dtype)
print("all equal to arithmetic modulo 256:", np.array_equal(got, want))
print("bits of 127, -128, -1:", [format(int(b), "08b") for b in v[[255, 0,
127]].view(np.uint8)])
```

```
pairs checked: 65536
result dtype:  int8
all equal to arithmetic modulo 256: True
bits of 127, -128, -1: ['01111111', '10000000', '11111111']
```

Floats. A `float32` has 1 sign bit, 8 exponent bits and 23 fraction bits. The value is `(-1)^sign × 2^(exponent − 127) × (1 + fraction / 2^23)`. The next block takes the bits of `0.1`, rebuilds the value with this formula, and compares it with what NumPy holds.

```
import numpy as np

bits = int(np.array([0.1], dtype=np.float32).view(np.uint32)[0])
sign, exponent, fraction = bits >> 31, (bits >> 23) & 0xFF, bits & 0x7FFFFF
rebuilt = (-1) ** sign * 2.0 ** (exponent - 127) * (1 + fraction / 2 ** 23)

print(f"sign={sign} exponent={exponent} fraction={fraction}")
print("rebuilt value:", rebuilt)
print("NumPy value:  ", float(np.float32(0.1)))
print("equal:", rebuilt == float(np.float32(0.1)))
print("gap above 1.0:       ", np.spacing(np.float32(1.0)))
print("gap above 16777216.0:", np.spacing(np.float32(16777216.0)))
```

```
sign=0 exponent=123 fraction=5033165
rebuilt value: 0.10000000149011612
NumPy value:   0.10000000149011612
equal: True
gap above 1.0:        1.1920929e-07
gap above 16777216.0: 2.0
```

The value is not 0.1. It is 0.10000000149011612, which is the nearest number that has 23 fraction bits. The gap between neighbor values is the value times 2^{-23}, so it doubles each time the exponent grows by 1. Above 2^{24} the gap is 2, and this is why `16777216 + 1` rounds back.

The same rule explains the cumulative sum from earlier in this chapter. A running total in `float32` must be a multiple of the gap at its size. Each add of 0.1 rounds to the nearest multiple of that gap. The next block measures the add of `float32(0.1)` at five sizes of the total.

```
import numpy as np

print(f"{'total':>9s} {'gap':>9s} {'0.1 became':>11s} {'change':>8s}")
for total in (1e5, 3e5, 5e5, 8e5, 1.2e6):
    t = np.float32(total)
    added = float((t + np.float32(0.1)) - t)
    print(f"{total:>9.0f} {float(np.spacing(t)):9.5f} {added:11.5f} {added / 0.1 - 1:+8.1%}")

x = np.full(10_000_000, 0.1, dtype=np.float32)
c = np.cumsum(x)
exact = float(np.float32(0.1)) * x.size
print("total passes 2**19 at step:", int(np.searchsorted(c, 2 ** 19)))
print("total passes 2**20 at step:", int(np.searchsorted(c, 2 ** 20)))
print("final relative error:      ", f"{float(c[-1]) / exact - 1:+.2%}")
```

```
    total       gap  0.1 became   change
   100000   0.00781     0.10156    +1.6%
   300000   0.03125     0.09375    -6.2%
   500000   0.03125     0.09375    -6.2%
   800000   0.06250     0.12500   +25.0%
  1200000   0.12500     0.12500   +25.0%
total passes 2**19 at step: 5490807
total passes 2**20 at step: 9685111
final relative error:       +8.79%
```

The add of 0.1 is not lost. It is rounded, and the rounding goes in both directions. At a total of 300,000 the gap is 0.03125, and 0.1 becomes 0.09375, which is 6 percent too small. At 800,000 the gap is 0.0625, and 0.1 becomes 0.125, which is 25 percent too large.

The total passes 2^{19} at step 5.5 million. Both sizes above 2^{19} in the table give an add that is 25 percent too large. Together with the steps below that size, this gives a final result that is 8.8 percent too large. The block did not measure each of the ten million adds, so the split between the ranges is not measured.

The invariant. For an integer dtype, every result is the true result modulo 2 to the number of bits. For a float dtype, every result is the nearest value that has the fraction bits of the type. You can test both statements, and the two blocks above do so.

The failure. Each wrong answer in this chapter has this cause. A `uint8` pixel of 200 plus 100 is 300 modulo 256, which is 44. A `float32` running total rounds every add to a multiple of its gap. The cumulative sum of ten million 0.1 values is 8.8 percent too large. NumPy does not report either one, because both results are correct under the rules of the dtype.

Reference

Callable	Introduced in	Contract
`np.ndarray.astype`	Integers wrap around	a copy with another dtype, under a casting rule
`np.iinfo`	The function you build	the limits of an integer type
`checked_add`	The function you build	integer add that raises `OverflowError` and never wraps
`np.sum`	A sum has its own dtype	total along an axis, in a type that you can set
`np.finfo`	Floats lose small additions	the limits and precision of a float type
`np.cumsum`	Floats lose small additions	running total, in a type that you can set
`np.can_cast`	Mixed dtypes: which one wins	whether a cast is allowed under a rule

What you have now

You have `checked_add`, which turns a silent wrap into an error. You also have four facts that you can check on any machine. A `uint8` add wraps. A `float32` cumulative sum can be wrong by 9 percent. The call `x.sum()` does not have that fault. A mixed expression has a dtype that you can print before you trust it.

The habit that follows is short. Print `.dtype` of every array at the edge of your code, where data comes in, and again where a result goes out.

Chapter 3 moves from what an array holds to who owns it. The next surprise is not a wrong value but a value that changed in a place you did not touch.

Chapter 3 — Views and copies

> **Build** — `root(a)`, a function that finds the array that owns the memory of any view, and a test that shows which operations copy and which do not.
>
> **Theory kept** — a slice is a new description of the same block. An index list or a mask makes a new block. A write through a view changes the original.
>
> **Source** — New. The rules are in the NumPy documentation under "Copies and views". Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch03_views`, which is `examples/ch03_views.py`.

Chapter 1 showed that `a.T` and `a[:, ::2]` read the block of `a` and copy nothing. This is the reason for the speed of NumPy. It is also the reason for one class of bug: a value that changed in a place that you did not touch.

This chapter gives the rule and builds a function that applies it. It shows two bugs that come from the rule. It also measures a cost that comes from the order of memory.

The rule

Basic slicing makes a view. Indexing with a list of integers or with a boolean mask makes a copy.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
view = a[:, 1:3]       # basic slice
fancy = a[[0, 2]]      # list of integers
masked = a[a > 5]      # boolean mask

print("slice  shares memory:", np.shares_memory(a, view))
print("list   shares memory:", np.shares_memory(a, fancy))
print("mask   shares memory:", np.shares_memory(a, masked))
```

```
slice  shares memory: True
list   shares memory: False
mask   shares memory: False
```

The reason is the strides from chapter 1. A slice is always a regular pattern, so one set of strides can describe it. A list of integers or a mask can pick any elements in any order, and no set of strides can describe that. NumPy must make a new block.

A write through a view changes the original.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
view = a[:, 1:3]
view[0, 0] = -1
print(a[0])
```

```
[ 0 -1  2  3]
```

The value at `a[0, 1]` is now −1, although the code never wrote to `a`.

The function you build

The `.base` attribute of a view points to the array that owns the block. NumPy sets it to the owner directly, even for a view of a view, so one step is enough. The loop below follows the chain to its end anyway. A block can belong to an object that is not an array, such as `bytes`.

```
import numpy as np


def root(a: np.ndarray) -> np.ndarray:
    """Follow .base links to the array that owns the memory of `a`."""
    while isinstance(a.base, np.ndarray):
        a = a.base
    return a


a = np.arange(12).reshape(3, 4)
b = a[1:]
c = b[:, 1:]
d = c[[0]]
print("c is a view of:", root(c) is root(a))
print("d is a view of:", root(d) is root(a))
print("root of d is d:", root(d) is d)
```

```
c is a view of: True
d is a view of: False
root of d is d: True
```

`root`

```
root(a: numpy.ndarray) -> numpy.ndarray
```

The function returns the array that owns the memory of `a`. If `a` owns its memory, it returns `a`.

parameter	default	what it does	what goes wrong
`a`	required	any `ndarray`, a view or an owner	an array built on `bytes` has a base that is not an array, so `root` returns that array and not the `bytes` object

The check `isinstance(a.base, np.ndarray)` is needed. An array can be built on top of bytes, a memory-mapped file, or a buffer from another library. In those cases `.base` is not an array, and the loop must stop.

Two arrays share memory when `root(x) is root(y)`. This is a fast test. It is not exact, because two views of one block can still cover different bytes. For the exact answer, use `np.shares_memory`, and see the next block.

```
import numpy as np

a = np.arange(10)
even, odd = a[::2], a[1::2]
print("may_share_memory:", np.may_share_memory(even, odd))
print("shares_memory:   ", np.shares_memory(even, odd))
```

```
may_share_memory: True
shares_memory:    False
```

The function `np.may_share_memory` compares only the first and last byte that each array covers. The two interleaved slices cover the same range, so it says yes. The function `np.shares_memory` solves the exact problem and says no. The exact function can be slow for complex strides.

The quick function can say yes when the exact answer is no. The next block tests the other direction on 5000 random pairs of slices: does the quick function ever say no when the exact answer is yes?

```
import numpy as np

rng = np.random.default_rng(0)
base = np.arange(60)


def random_slice():
    start = int(rng.integers(0, 60))
    stop = int(rng.integers(start, 61))
    step = int(rng.integers(1, 6))
    return base[start:stop:step]


both = quick_yes_exact_no = quick_no_exact_yes = 0
for _ in range(5000):
    a, b = random_slice(), random_slice()
    if a.size == 0 or b.size == 0:
        continue
    exact, quick = np.shares_memory(a, b), np.may_share_memory(a, b)
    both += exact and quick
    quick_yes_exact_no += quick and not exact
    quick_no_exact_yes += exact and not quick

print("both say yes:                    ", both)
print("quick says yes, exact says no:   ", quick_yes_exact_no)
print("quick says no, exact says yes:   ", quick_no_exact_yes)
```

```
both say yes:                    1634
quick says yes, exact says no:   581
quick says no, exact says yes:   0
```

The third count is 0, as the documentation promises. A `False` from the quick function is certain. A `True` is not.

`np.may_share_memory`

```
np.may_share_memory(a, b, /, max_work=0)
```

The function is the quick version of `np.shares_memory`. It tests only whether the memory ranges of the two arrays overlap.

parameter	default	what it does	what goes wrong
`a`, `b`	required	the two arrays to compare	—
`max_work`	`0`	the effort. `0` compares only the first and last byte of each array	the interleaved slices `a[::2]` and `a[1::2]` give `True`, and `np.shares_memory` gives `False`

Bug 1: a function that changes its argument

A function that uses `-=` on its argument writes into the memory of the caller.

```
import numpy as np


def center(x):
    x -= x.mean()
    return x


data = np.array([[1.0, 10.0], [3.0, 20.0], [5.0, 30.0]])
col = data[:, 0]
centered = center(col)
print("data after the call:")
print(data)
```

```
data after the call:
[[-2. 10.]
 [ 0. 20.]
 [ 2. 30.]]
```

The caller passed one column. The column is a view, so the call changed the first column of `data`. The caller sees a different table, and the function did not return it.

The fix is one character: `x = x - x.mean()` makes a new array. The in-place operators `+=`, `-=`, `*=` and `/=` are faster and use less memory. Use them on arrays that your function made itself. Do not use them on an argument unless the name or the documentation says that the function changes it.

Bug 2: a small view that keeps a large block alive

A view keeps its block alive. If you slice 10 rows from a 288 MB array and then delete the large array, the 288 MB stays in memory.

```
import numpy as np

big = np.zeros((6000, 6000))
small = big[:10]
print("small is", small.nbytes, "bytes")
print("small keeps", small.base.nbytes, "bytes alive")

small = small.copy()
print("after copy, base is:", small.base)
```

```
small is 480000 bytes
small keeps 288000000 bytes alive
after copy, base is: None
```

Use `.copy()` when you keep a small piece of a large array. The copy owns 10 rows, and the large array is free to go.

What a copy costs, and what a bad walk costs

Chapter 1 said that a transpose is not in C order. A C-contiguous array has its last axis in consecutive memory. NumPy reads consecutive memory fast. A likely cause is the line of 64 bytes that one read brings in. A strided read wastes most of that line. This book did not test the explanation.

Measure two cases on a 6000-by-6000 `float64` array, which is 288 MB. The first is a plain copy against a copy of the transpose. The second is a sum over 600 rows against a sum over 600 columns.

```
import timeit

import numpy as np

n = 6000
big = np.random.default_rng(0).random((n, n))


def best(f, repeat=3):
    return min(timeit.repeat(f, number=1, repeat=repeat))


t_copy = best(lambda: big.copy())
t_tcopy = best(lambda: np.ascontiguousarray(big.T))
t_row = best(lambda: [big[j, :].sum() for j in range(0, n, 10)])
t_col = best(lambda: [big[:, j].sum() for j in range(0, n, 10)])
t_ax0 = best(lambda: big.sum(axis=0))
t_ax1 = best(lambda: big.sum(axis=1))

assert t_tcopy / t_copy > 1.3
assert t_col / t_row > 3
assert 0.33 < t_ax0 / t_ax1 < 3
print(f"copy of transpose / plain copy   {t_tcopy / t_copy:5.1f}x")
print(f"600 column sums / 600 row sums   {t_col / t_row:5.1f}x")
print(f"sum(axis=0) / sum(axis=1)        {t_ax0 / t_ax1:5.1f}x")
```

```
# output varies
copy of transpose / plain copy     3.0x
600 column sums / 600 row sums    15.8x
sum(axis=0) / sum(axis=1)          1.7x
```

The ratios above are from one run on a shared machine. The asserts are the claims. The transposed copy is at least 1.3 times slower. The column loop is at least 3 times slower. The two whole-array sums are within a factor of 3 of each other.

The third line is a result that you can find surprising. The sum along axis 0 walks against the memory order, and it is still not slow. NumPy does not add one column at a time. It adds whole rows into a row of partial sums, which is the order that memory has. The built-in reductions know the layout. Your own Python loop over columns does not.

`np.ascontiguousarray`

```
np.ascontiguousarray(a, dtype=None, *, like=None)
```

The function returns an array in C order. It copies only when the input is not already C-contiguous.

parameter	default	what it does	what goes wrong
`a`	required	any array or array-like	for an input that is already contiguous, the result is the input itself, and a write reaches the original
`dtype`	`None`	the type of the result. With `None`, the dtype of the input	—

The rule that follows has two parts. Use the built-in reduction when one exists. When you must loop in Python, loop over the first axis, so that each step reads one consecutive row.

Which operations copy

Two methods decide the first rows of the table, so their signatures come first.

`np.ndarray.reshape`

```
np.ndarray.reshape(self, /, *shape, order='C', copy=None)
```

The method returns an array with the same values and a new shape. It returns a view when the strides allow it.

parameter	default	what it does	what goes wrong
`shape`	required	the new shape, as a tuple or as separate integers. One length can be `-1`, and NumPy computes it	the product of the new lengths must equal the size, or the call raises `ValueError`
`order`	`'C'`	the order in which values are read and placed. `'C'` means that the last axis changes fastest	`'F'` on a C array reorders the values, and it can force a copy
`copy`	`None`	`None` gives a view when possible and a copy when not. `True` always copies. `False` raises `ValueError` if a copy cannot be avoided	with `None` you do not know if a later write reaches the original. With `False` the transposed case raises `Unable to avoid creating a copy while reshaping.`

`np.ndarray.copy`

```
np.ndarray.copy(self, /, order='C')
```

The method returns a new array with its own block.

parameter	default	what it does	what goes wrong
`order`	`'C'`	the memory order of the copy. `'F'` is Fortran order. `'A'` is the order of the input. `'K'` is as close to the input as possible	the function `np.copy` has the default `'K'`, so `a.T.copy()` has strides `(16, 8)` and `np.copy(a.T)` has strides `(8, 24)` for the array in the next block

The code below tests each line of the summary table that follows. It raises an error if one line is wrong.

```
import numpy as np

a = np.arange(12).reshape(3, 4)
t = a.T
rows = {
    "a[1:3]":            (a[1:3], True),
    "a[:, ::2]":         (a[:, ::2], True),
    "a.T":               (t, True),
    "a[[0, 2]]":         (a[[0, 2]], False),
    "a[a > 5]":          (a[a > 5], False),
    "a.reshape(4, 3)":   (a.reshape(4, 3), True),
    "a.T.reshape(12)":   (t.reshape(12), False),
    "a.ravel()":         (a.ravel(), True),
    "a.T.ravel()":       (t.ravel(), False),
    "a.flatten()":       (a.flatten(), False),
    "a.copy()":          (a.copy(), False),
    "np.array(a)":       (np.array(a), False),
}
for name, (result, expected) in rows.items():
    shared = np.shares_memory(a, result)
    assert shared == expected, name
    print(f"{name:18s} {'view' if shared else 'copy'}")
```

```
a[1:3]             view
a[:, ::2]          view
a.T                view
a[[0, 2]]          copy
a[a > 5]           copy
a.reshape(4, 3)    view
a.T.reshape(12)    copy
a.ravel()          view
a.T.ravel()        copy
a.flatten()        copy
a.copy()           copy
np.array(a)        copy
```

Operation	Result
`a[1:3]`, `a[:, ::2]`, `a.T`	view
`a[[0, 2]]`, `a[a > 5]`	copy
`a.reshape(...)`	view when strides allow, copy otherwise
`a.ravel()`	view when the array is C-contiguous, copy otherwise
`a.flatten()`, `a.copy()`, `np.array(a)`	always a copy

The call `a.reshape(...)` can change from a view to a copy when the array changes layout. Code that depends on the write-through must not rely on `reshape`. Test with `np.shares_memory` or use `np.may_share_memory`, and do not guess.

How it works

A slice makes a new array object, not a new block. The object holds a pointer into the block, a shape, strides and a few flags. Making it costs the same for a small array and for a large one.

The next block measures this. It makes a view of 5000 rows of a 288 MB array and a copy of the whole array.

```
import timeit

import numpy as np

big = np.zeros((6000, 6000))
t_view = min(timeit.repeat(lambda: big[:5000], number=1000, repeat=3)) / 1000
t_copy = min(timeit.repeat(lambda: big.copy(), number=1, repeat=3))
assert t_copy / t_view > 1000
print("view is a new object over the same block:", big[:5000].base is big)
print("view costs under a millisecond:", t_view < 1e-3)
print("a copy costs at least 1000 times a view:", t_copy / t_view > 1000)
```

```
view is a new object over the same block: True
view costs under a millisecond: True
a copy costs at least 1000 times a view: True
```

The flags show what the object knows about its block. The `OWNDATA` flag tells who frees the memory. The `WRITEABLE` flag tells if a write is allowed.

```
import numpy as np

a = np.arange(12, dtype=np.int32).reshape(3, 4)
cases = {
    "a": a,
    "a.T": a.T,
    "a[::2]": a[::2],
    "broadcast_to": np.broadcast_to(np.arange(3.0), (2, 3)),
}
print(f"{'':14s} {'C':>5s} {'F':>5s} {'write':>6s} {'owns':>6s}")
for name, v in cases.items():
    f = v.flags
    print(f"{name:14s} {f.c_contiguous!s:>5s} {f.f_contiguous!s:>5s} {f.writeable!s:>6s}
{f.owndata!s:>6s}")

try:
    cases["broadcast_to"][0, 0] = 1.0
except ValueError as e:
    print("write to broadcast_to:", e)

frozen = a.copy()
frozen.setflags(write=False)
try:
    frozen[0, 0] = 1
except ValueError as e:
    print("write to frozen copy: ", e)
```

```
                   C     F  write   owns
a               True False   True  False
a.T            False  True   True  False
a[::2]         False False   True  False
broadcast_to   False False  False  False
write to broadcast_to: assignment destination is read-only
write to frozen copy:  assignment destination is read-only
```

The result of `np.broadcast_to` is read-only. The NumPy documentation gives the reason: many of its elements share one memory location. A write to one element changes all of them.

You can protect your own arrays in the same way with `setflags(write=False)`. A function that must not change its argument can take a read-only view. A write then fails at once with an error. It does not give a wrong number later.

The invariant. A view and its owner read the same bytes. A write through either one is visible through the other, and `np.shares_memory` returns `True`.

The failure. A function that writes into its argument changes the data of the caller, as bug 1 showed. The cost is not a wrong number in the function. It is a wrong number somewhere else, later, with no link to the cause.

Reference

Callable	Introduced in	Contract
`root`	The function you build	the array that owns the memory of a view
`np.may_share_memory`	The function you build	quick test: do the memory ranges overlap
`np.ascontiguousarray`	What a copy costs, and what a bad walk costs	C-ordered array, copied only when needed
`np.ndarray.reshape`	Which operations copy	new shape, a view when the strides allow
`np.ndarray.copy`	Which operations copy	a new block, in a given memory order

What you have now

You have `root(a)`, and you can say which of your arrays share memory. You saw two bugs that come from views. A function changed its argument. A small view kept 288 MB alive.

You also measured the cost of a walk against the memory order. The asserts prove two bounds. A copy of the transpose is at least 1.3 times slower. A Python loop over columns is at least 3 times slower. The built-in reductions do not have this fault.

Chapter 4 uses the other half of the rule. Indexing with lists and masks makes copies, and writing through them has its own trap.

Chapter 4 — Indexing

> **Build** — `group_sum(keys, values, k)`, which adds up the values for each key, in three versions that you can time against each other.
>
> **Theory kept** — an index with integers or a mask picks elements and makes a copy. A write with an index list applies to each position once. A condition in `np.where` is computed on both sides.
>
> **Source** — New. The behavior is from the NumPy documentation on advanced indexing. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch04_indexing`, which is `examples/ch04_indexing.py`.

Chapter 3 said that a list of integers or a mask makes a copy. This chapter shows the four traps that follow from that rule. Each trap produces a wrong result with no error. The chapter then builds a function that counts by key, which is the most common reason for an index with repeated values.

Reading with a mask

A mask is a boolean array of the same shape as the data. A comparison makes one. The mask can count, select and replace.

```
import numpy as np

x = np.array([3, -1, 4, -1, 5, -9, 2, 6])
mask = x < 0
print("mask:    ", mask)
print("count:   ", mask.sum())
print("selected:", x[mask])
print("fraction:", mask.mean())
```

```
mask:     [False  True False  True False  True False False]
count:    3
selected: [-1 -1 -9]
fraction: 0.375
```

The sum of a boolean array counts the `True` values. The mean gives the fraction. Both use the fact that `True` is 1.

Trap 1: a write to a copy is lost

A mask on the left of `=` writes into the array. A mask on the right makes a copy, and a write into the copy is lost.

```
import numpy as np

m = np.arange(10)
m[m > 5][0] = 99
print("chained write:", m)

m[m > 5] = 99
print("direct write: ", m)
```

```
chained write: [0 1 2 3 4 5 6 7 8 9]
direct write:  [ 0  1  2  3  4  5 99 99 99 99]
```

The first line looks like it sets one element to 99. Python runs two steps: `m[m > 5]` makes a copy, and `[0] = 99` writes into that copy. The copy is then deleted. The array `m` did not change, and NumPy did not say so. The second line is one step, and it works.

Trap 2: a repeated index is applied once

Add 1 at each position in an index list. If one position appears three times, you want 3. You get 1.

```
import numpy as np

idx = np.array([1, 1, 3, 1])

a = np.zeros(5, dtype=int)
a[idx] += 1
print("a[idx] += 1  :", a)

b = np.zeros(5, dtype=int)
np.add.at(b, idx, 1)
print("np.add.at    :", b)

print("np.bincount  :", np.bincount(idx, minlength=5))
```

```
a[idx] += 1  : [0 1 0 1 0]
np.add.at    : [0 3 0 1 0]
np.bincount  : [0 3 0 1 0]
```

`np.add.at`

```
np.add.at(a, indices, b=None, /)
```

The method adds `b` to `a` at each position in `indices`, one element at a time. It changes `a` in place and returns `None`.

parameter	default	what it does	what goes wrong
`a`	required	the array to change. It is changed in place	a result that you expect from the call does not exist: the return value is `None`
`indices`	required	an index list, or a tuple of index lists for several axes. A position can appear many times	a negative index counts from the end, so `-1` changes the last element
`b`	`None`	the value to add. A scalar or an array that broadcasts to the indexed shape	—

`np.bincount`

```
np.bincount(x, /, weights=None, minlength=0)
```

The function counts how many times each integer occurs, or adds a weight for each occurrence.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of non-negative integers	a negative value raises `ValueError`
`weights`	`None`	one weight for each element of `x`. The result then holds the sum of the weights, as `float64`	the weights make the result `float64`, also when they are integers
`minlength`	`0`	the least length of the result	without it, the result has length `max(x) + 1`, and it can differ between two runs on different data

The statement `a[idx] += 1` is `a[idx] = a[idx] + 1`. NumPy reads `a[idx]` once, adds 1 to each of the four values, and writes four results. Position 1 receives the same result three times. The function `np.add.at` goes one element at a time, so it is correct. The function `np.bincount` counts directly.

Trap 3: `np.where` computes both sides

`np.where`

```
np.where(condition, x=None, y=None, /)
```

With three arguments, the function picks from `x` where the condition is `True` and from `y` where it is `False`. With one argument, it returns the positions where the condition is `True`.

parameter	default	what it does	what goes wrong
`condition`	required	a boolean array, or an array that NumPy reads as boolean	a count of 0 and 1 values is read as a condition: `0` is `False` and every other value is `True`
`x`	`None`	the values where the condition is `True`	it is computed for every element, also the ones that the condition rejects
`y`	`None`	the values where the condition is `False`	it is computed for every element, also the ones that the condition accepts

`np.where(cond, x, y)` picks values from `x` or from `y`. Both `x` and `y` are computed first. A protection that you put in the condition does not stop a bad value in the branch.

```
import warnings

import numpy as np

x = np.array([4.0, 0.0, 2.0])

with warnings.catch_warnings(record=True) as caught:
    warnings.simplefilter("always")
    r = np.where(x != 0, 1 / x, 0.0)
print("result:  ", r)
print("warnings:", [str(w.message) for w in caught])

r2 = np.divide(1, x, out=np.zeros_like(x), where=x != 0)
print("with where=:", r2)
```

```
result:   [0.25 0.   0.5 ]
warnings: ['divide by zero encountered in divide']
with where=: [0.25 0.   0.5 ]
```

The result is right and the warning shows that the division by zero ran. For division the harm is small, because the bad value is not selected. For `np.log` of a negative number, the branch that you did not select gives a warning. For an index that is out of range, it raises an error. The `where=` argument of a ufunc skips the elements where the condition is false. Chapter 6 covers this argument.

Trap 4: the shape of a mixed index

When you mix a slice and an index list in one index, NumPy can move the axes of the result.

```
import numpy as np

A = np.arange(24).reshape(2, 3, 4)
print(A[0, :, [0, 1]].shape)
print(A[:, [0, 1], [0, 1]].shape)
print(A[0][:, [0, 1]].shape)
```

```
(2, 3)
(2, 2)
(3, 2)
```

Read the first line. The index picks `A[0]` and then columns 0 and 1, and you can expect the shape `(3, 2)`. The result is `(2, 3)`. The rule is that the axes that come from index lists go first when a slice separates two advanced indices. In the first line the scalar `0` and the list `[0, 1]` are both advanced indices, and the slice sits between them.

The third line avoids the problem. It indexes in two steps, and the shape is `(3, 2)`. When the shape of a mixed index matters, use two steps or print `.shape` before you rely on it.

The function you build

A group-by sum is the most common use of a repeated index. You have one value per row and one key per row, and you want one total per key. Three versions follow. They must give the same answer.

```
import numpy as np


def group_sum_loop(keys: np.ndarray, values: np.ndarray, k: int) -> np.ndarray:
    """Add up `values` for each key in 0..k-1 with a Python loop."""
    out = [0.0] * k
    for key, v in zip(keys.tolist(), values.tolist()):
        out[key] += v
    return np.array(out)


def group_sum_at(keys: np.ndarray, values: np.ndarray, k: int) -> np.ndarray:
    """Add up `values` for each key in 0..k-1 with np.add.at."""
    out = np.zeros(k)
    np.add.at(out, keys, values)
    return out


def group_sum_bincount(keys: np.ndarray, values: np.ndarray, k: int) -> np.ndarray:
    """Add up `values` for each key in 0..k-1 with np.bincount."""
    return np.bincount(keys, weights=values, minlength=k)


rng = np.random.default_rng(0)
n, k = 1_000_000, 100
keys = rng.integers(0, k, n)
values = rng.random(n)

ref = group_sum_bincount(keys, values, k)
print("loop    matches bincount:", np.allclose(group_sum_loop(keys, values, k), ref))
print("add.at  matches bincount:", np.allclose(group_sum_at(keys, values, k), ref))
print("total of all groups     :", round(float(ref.sum()), 3), "=", round(float(values.sum()),
3))
```

```
loop    matches bincount: True
add.at  matches bincount: True
total of all groups     : 500316.548 = 500316.548
```

The third line is a second check that does not depend on the other functions. The total of the groups must equal the total of all values.

Now time the three versions.

```
import timeit

import numpy as np

rng = np.random.default_rng(0)
n, k = 1_000_000, 100
keys = rng.integers(0, k, n)
values = rng.random(n)


def loop():
    out = [0.0] * k
    for key, v in zip(keys.tolist(), values.tolist()):
        out[key] += v


def at():
    out = np.zeros(k)
    np.add.at(out, keys, values)


def bincount():
    np.bincount(keys, weights=values, minlength=k)


t = {f.__name__: min(timeit.repeat(f, number=1, repeat=3)) for f in (loop, at, bincount)}
assert t["loop"] / t["at"] > 5
assert t["loop"] / t["bincount"] > 5
assert 0.5 < t["at"] / t["bincount"] < 2          # the two NumPy versions are within a factor
of 2
for name, sec in t.items():
    print(f"{name:9s} {sec * 1e3:8.2f} ms")
```

```
# output varies
loop        118.63 ms
at            2.66 ms
bincount      2.96 ms
```

The assert proves that the loop is more than 5 times slower than either NumPy version. That is the expected result. The surprise is the second line against the third line.

Old advice says that `np.add.at` is very slow and that you must use `np.bincount`. That advice dates from NumPy 1.x, which this book did not install, so it cannot say whether it was true there. On NumPy 2.5.4 the assert proves that the two are within a factor of 2 of each other on this input.

Check the version of NumPy before you accept advice about speed. The `bincount` version has one restriction. The keys must be non-negative integers. The function `np.add.at` accepts any index that is valid for the output array.

`group_sum_loop`

```
group_sum_loop(keys: numpy.ndarray, values: numpy.ndarray, k: int) -> numpy.ndarray
```

The function adds the values of each group with a Python loop. It is the definition, and the other two are checked against it.

parameter	default	what it does	what goes wrong
`keys`	required	one non-negative integer for each row, in `0 .. k-1`	a key of `k` or more raises `IndexError`
`values`	required	one number for each row, with the same length as `keys`	`zip` stops at the shorter input, so a length mismatch gives no error
`k`	required	the number of groups, which is the length of the result	—

`group_sum_at`

```
group_sum_at(keys: numpy.ndarray, values: numpy.ndarray, k: int) -> numpy.ndarray
```

The function adds the values with `np.add.at`, which handles a repeated key correctly.

parameter	default	what it does	what goes wrong
`keys`	required	one integer for each row, in `0 .. k-1`	a key of `k` or more raises `IndexError`. A key of `-1` adds to the last group
`values`	required	one number for each row, with the same length as `keys`	a different length raises an error
`k`	required	the number of groups, which is the length of the result	—

`group_sum_bincount`

```
group_sum_bincount(keys: numpy.ndarray, values: numpy.ndarray, k: int) -> numpy.ndarray
```

The function adds the values with `np.bincount`, which accepts only non-negative integer keys.

parameter	default	what it does	what goes wrong
`keys`	required	one non-negative integer for each row	a negative key raises `ValueError`
`values`	required	one number for each row, with the same length as `keys`	a different length raises an error
`k`	required	the least length of the result	a key of `k` or more makes the result longer than `k`, with no error

Finding the largest values without a sort

A full sort puts every value in order. To find the 10 largest of 5 million values, you do not need that. `np.partition` puts the 10 largest at the end, in any order, and it does less work.

```
import timeit

import numpy as np

x = np.random.default_rng(1).random(5_000_000)
top_sort = np.sort(x)[-10:]
top_part = np.sort(np.partition(x, -10)[-10:])
print("same ten values:", np.array_equal(top_sort, top_part))

t_sort = min(timeit.repeat(lambda: np.sort(x)[-10:], number=1, repeat=3))
t_part = min(timeit.repeat(lambda: np.partition(x, -10)[-10:], number=1, repeat=3))
assert t_sort / t_part > 2
print(f"sort / partition: {t_sort / t_part:.1f}x")
```

```
# output varies
same ten values: True
sort / partition: 3.0x
```

The partition result is not in order, so the code sorts the 10 values at the end. The assert requires a saving of at least 2 times.

`np.partition`

```
np.partition(a, kth, axis=-1, kind='introselect', order=None)
```

The function puts the `kth` smallest value in its sorted position. It puts smaller values before it and larger values after it, in no order.

parameter	default	what it does	what goes wrong
`a`	required	the array to partition. The function returns a copy	the input is not changed, and the result is a new block
`kth`	required	the index of the value to put in place. A negative index counts from the end, so `-10` puts the 10 largest at the end	the values on each side are not sorted
`axis`	`-1`	the axis to partition along	—
`kind`	`'introselect'`	the selection algorithm. NumPy has one	—

How it works

An index with integer arrays or a mask is a gather. NumPy broadcasts the index arrays against each other. It reads one element for each position of the broadcast shape. It writes the elements into a new array. This explains the four traps, and the next blocks test each part.

First, a mask is a list of integer indices in disguise. NumPy turns the mask into the positions of its `True` values, then gathers.

```
import numpy as np

rng = np.random.default_rng(0)
m = rng.random((4, 5))
mask = m > 0.5
positions = np.nonzero(mask)

print("mask picks the same values as the positions:", np.array_equal(m[mask], m[positions]))
print("positions are two arrays, one for each axis:", [p.shape for p in positions])
```

```
mask picks the same values as the positions: True
positions are two arrays, one for each axis: [(12,), (12,)]
```

Second, the shape of the result is the broadcast shape of the index arrays. A column of row numbers and a row of column numbers give a grid, as in chapter 5.

```
import numpy as np

m = np.arange(20).reshape(4, 5)
rows = np.array([[0], [2]])
cols = np.array([1, 3, 4])

picked = m[rows, cols]
print("index shapes:", rows.shape, cols.shape)
print("broadcast shape:", np.broadcast_shapes(rows.shape, cols.shape))
print("result shape:", picked.shape)
print(picked)
```

```
index shapes: (2, 1) (3,)
broadcast shape: (2, 3)
result shape: (2, 3)
[[ 1  3  4]
 [11 13 14]]
```

Third, a write is a scatter. If a position appears more than once, the last value stays, and an in-place add reads the old value only once. The next block spells out `a[idx] += 1` in three steps.

```
import numpy as np

v = np.array([0, 10, 20, 30, 40])
idx = np.array([1, 1, 3])

fast = v.copy()
fast[idx] += 1

slow = v.copy()
old = slow[idx]          # step 1: gather. Position 1 is read twice, and both reads give 10
new = old + 1            # step 2: add. Both copies become 11
slow[idx] = new          # step 3: scatter. Position 1 is written twice with 11

print("same as the one-line form:", np.array_equal(fast, slow))
print("result:", fast, "and position 1 went up by 1, not 2")

last = np.zeros(2)
last[[0, 0]] = [1.0, 2.0]
print("repeated index in a plain write, the last value stays:", last)
```

```
same as the one-line form: True
result: [ 0 11 20 31 40] and position 1 went up by 1, not 2
repeated index in a plain write, the last value stays: [2. 0.]
```

Fourth, `np.take` is the gather without the broadcast rule, and it has a `mode` for positions out of range. The indexing form raises `IndexError` for a position out of range. The method `np.take` can clip.

```
import numpy as np

x = np.arange(3)
try:
    x[[5]]
except IndexError as e:
    print("x[[5]]:", e)
print("take, mode='clip':", x.take([5], mode="clip"))
print("take, mode='wrap':", x.take([5], mode="wrap"))
```

```
x[[5]]: index 5 is out of bounds for axis 0 with size 3
take, mode='clip': [2]
take, mode='wrap': [2]
```

The position 5 is clipped to the last element, 2, or wrapped to `5 % 3`, which is 2 as well. Both modes hide an index that is wrong. Use them only when that is what you mean.

The invariant. The result of an index with arrays has the broadcast shape of the index arrays, and it is a new block. The result of a basic slice is a view.

The failure. The write `a[idx] += 1` is a gather, an add and a scatter. A repeated index loses all but one of its updates. The block above shows position 1 going up by 1. The loop `for i in idx: a[i] += 1` adds 2.

Reference

Callable	Introduced in	Contract
`np.add.at`	Trap 2: a repeated index is applied once	add at positions, one at a time, in place
`np.where`	Trap 3: `np.where` computes both sides	pick from two arrays by a condition
`np.bincount`	Trap 2: a repeated index is applied once	count or sum weights for each non-negative integer
`group_sum_loop`	The function you build	group sum, the definition
`group_sum_at`	The function you build	group sum with `np.add.at`
`group_sum_bincount`	The function you build	group sum with `np.bincount`
`np.partition`	Finding the largest values without a sort	the k-th value in place, the rest unordered

What you have now

You have three versions of `group_sum` that give the same answer, and you know which one to use first: `np.bincount` when the keys are small non-negative integers, and `np.add.at` for the other cases. You know four traps by name.

- A chained write is lost.
- A repeated index is applied once.
- The `np.where` function computes both sides.
- A mixed index moves axes.

Chapter 5 uses indexing in the other direction. It does not pick elements from an array. It stretches arrays so that they combine without any loop.

Chapter 5 — Broadcasting

> **Build** — three functions that compute all distances between two sets of points, a loop version, a broadcast version and a dot-product version, with a measured memory cost and a measured precision limit for each.
>
> **Theory kept** — two shapes combine when, from the right, each pair of lengths is equal or one of them is 1. A length of 1 is stretched with a stride of 0, so the stretch costs no memory until an operation writes a result.
>
> **Source** — New. The rule is in the NumPy documentation under "Broadcasting". Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch05_broadcasting`, which is `examples/ch05_broadcasting.py`.

Broadcasting is the rule that lets `a - b` work when `a` and `b` have different shapes. It replaces most of the loops that you write in plain Python. It also makes one new mistake easy: a result that is much larger than you meant.

This chapter states the rule, shows the stride trick behind it, and uses it to compute all pairwise distances. The distance example is the right size to show the three costs of a vectorized program: time, memory and precision.

The rule

Write the two shapes one above the other, aligned on the right. For each column, the two lengths must be equal, or one of them must be 1. A missing length on the left counts as 1. The result has the larger length in each column.

```
import numpy as np

print(np.broadcast_shapes((3, 1), (1, 4)))
print(np.broadcast_shapes((8, 1, 6, 1), (7, 1, 5)))
print(np.broadcast_shapes((5, 3), (3,)))

try:
    np.ones(3) + np.ones(4)
except ValueError as e:
    print("error:", e)
```

```
(3, 4)
(8, 7, 6, 5)
(5, 3)
error: operands could not be broadcast together with shapes (3,) (4,)
```

Read the second line with the rule. The shapes `(8, 1, 6, 1)` and `(7, 1, 5)` align as columns `8 1 6 1` over `_ 7 1 5`. The columns pair as 1 with 5, 6 with 1, 1 with 7, and 8 with nothing. The result is `(8, 7, 6, 5)`. The last line shows what happens when two lengths differ and neither is 1.

`np.broadcast_shapes`

```
np.broadcast_shapes(*args)
```

The function returns the shape that the rule gives for the input shapes. It makes no array, so it is a cheap test.

parameter	default	what it does	what goes wrong
`args`	required	two or more shapes, each a tuple of integers	when two lengths differ and neither is 1, it raises `ValueError` and names the shapes

Use `None` or `np.newaxis` to add a length of 1 at any position. This is the tool that you use to line up the axes that you want.

```
import numpy as np

x = np.arange(3)
print((x[:, None] - x[None, :]))
print((x[:, None] - x[None, :]).shape)
```

```
[[ 0 -1 -2]
 [ 1  0 -1]
 [ 2  1  0]]
(3, 3)
```

The column vector `x[:, None]` has shape `(3, 1)`, and the row vector `x[None, :]` has shape `(1, 3)`. The result `(3, 3)` holds every difference `x[i] - x[j]`.

A stretch costs no memory

NumPy does not copy a length-1 axis to make it longer. It sets the stride of that axis to 0, so every step reads the same element. This is the idea from chapter 1, used again.

```
import numpy as np

row = np.arange(3.0)
stretched = np.broadcast_to(row, (1000, 3))
print("shape  ", stretched.shape)
print("strides", stretched.strides)
print("block  ", stretched.base.nbytes, "bytes")
print("result of an add:", (stretched + 1).nbytes, "bytes")
```

```
shape   (1000, 3)
strides (0, 8)
block   24 bytes
result of an add: 24000 bytes
```

The stretched array looks like 1000 rows. It has a block of 24 bytes and a stride of 0 along the first axis. The cost appears when an operation makes a result, because the result must have all 1000 rows.

`np.broadcast_to`

```
np.broadcast_to(array, shape, subok=False)
```

The function returns a read-only view with the given shape. It makes the stride of each stretched axis 0.

parameter	default	what it does	what goes wrong
`array`	required	the array to stretch	—
`shape`	required	the target shape. The rule of this chapter must give it from the shape of `array`	a shape that the rule does not allow raises `ValueError`
`subok`	`False`	with `True`, a subclass of `ndarray` passes through	—

A write to the result raises `ValueError: assignment destination is read-only`. Many of its elements share one memory location, and a write changes all of them.

The trap: a result you did not mean

A vector of shape `(n,)` and a column of shape `(n, 1)` broadcast to `(n, n)`. NumPy gives no warning, because the rule is satisfied.

```
import numpy as np

rng = np.random.default_rng(0)
values = rng.random(1000)
column = rng.random((1000, 1))

r = values - column
print("shape:", r.shape)
print("size: ", r.nbytes / 1e6, "MB")

X = rng.random((5, 3))
try:
    X - X.mean(axis=1)
except ValueError as e:
    print("error:", e)
print("with keepdims:", (X - X.mean(axis=1, keepdims=True)).shape)
```

```
shape: (1000, 1000)
size:  8.0 MB
error: operands could not be broadcast together with shapes (5,3) (5,)
with keepdims: (5, 3)
```

For `n = 1000` the mistake costs 8 MB, and nobody notices. For `n = 100,000` the same mistake needs 80 GB. The usual cause is a function that returned `(n, 1)` where you expected `(n,)`. An example is a `reshape(-1, 1)` that you added for another library.

The third part of the block is the correct way to subtract a row mean. The call `X.mean(axis=1)` has shape `(5,)`, which does not align with `(5, 3)`. The argument `keepdims=True` keeps the length-1 axis, so the shape is `(5, 1)` and the subtraction works.

`np.mean`

```
np.mean(a, axis=None, dtype=None, out=None, keepdims=<no value>, *, where=<no value>)
```

The function returns the mean along an axis. The method `a.mean(...)` takes the same arguments after `self`.

parameter	default	what it does	what goes wrong
`a`	required	the array	an empty array gives `nan` and a warning
`axis`	`None`	the axis to average along. With `None`, all values	`axis=1` on a `(5, 3)` array gives shape `(5,)`, which does not align with `(5, 3)`
`keepdims`	`<no value>`	with `True`, the averaged axis stays with length 1	without it, the subtraction of a row mean from the array is an error

The habit that follows: assert the shape of a result at a boundary. A line such as `assert r.shape == (n,)` is cheap and catches this mistake at once.

The function you build

Given `A` with `n` points and `B` with `m` points, in `d` dimensions, build the `(n, m)` matrix of distances. Three versions follow.

```
import numpy as np


def dist_loop(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Distances between every row of A and every row of B, with two Python loops."""
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Exact. Builds an (n, m, d) temporary array."""
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Fast and small, but loses precision when points are close together and far from the
origin."""
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((300, 16)), rng.random((400, 16))
ref = dist_loop(A, B)
print("shape:", ref.shape)
print("broadcast max error:", np.abs(dist_broadcast(A, B) - ref).max())
print("dot       max error:", np.abs(dist_dot(A, B) - ref).max())
```

```
shape: (300, 400)
broadcast max error: 0.0
dot       max error: 2.886579864025407e-15
```

The broadcast version is the same formula as the loop, with `A[:, None, :]` of shape `(n, 1, d)` and `B[None, :, :]` of shape `(1, m, d)`. It subtracts to `(n, m, d)`, squares, and sums over the last axis. The dot version uses the identity that the squared distance is `|a|² + |b|² − 2 a·b`. The cross term `A @ B.T` is one matrix product.

On this well-behaved input, all three agree. The dot version differs from the loop by about 3e-15, which is the rounding of `float64`.

The four versions have one signature. They differ in what goes wrong, and each of the next sections measures one of those costs.

`dist_loop`

```
dist_loop(A: numpy.ndarray, B: numpy.ndarray) -> numpy.ndarray
```

The function returns the `(n, m)` matrix of distances with two Python loops. It is the definition.

parameter	default	what it does	what goes wrong
`A`	required	`n` points as an `(n, d)` array	—
`B`	required	`m` points as an `(m, d)` array, with the same `d` as `A`	a different `d` raises `ValueError`

`dist_broadcast`

```
dist_broadcast(A: numpy.ndarray, B: numpy.ndarray) -> numpy.ndarray
```

The function returns the same matrix with one broadcast subtraction.

parameter	default	what it does	what goes wrong
`A`	required	`n` points as an `(n, d)` array	the temporary array has `n * m * d` values: 2080 MB for 2000 points of 64 dimensions
`B`	required	`m` points as an `(m, d)` array, with the same `d` as `A`	a different `d` raises `ValueError`

`dist_dot`

```
dist_dot(A: numpy.ndarray, B: numpy.ndarray) -> numpy.ndarray
```

The function returns the same matrix from norms and one matrix product.

parameter	default	what it does	what goes wrong
`A`	required	`n` points as an `(n, d)` array	for points in a cube of side 0.01 at 10,000: error 3.5e-4 in `float64`. In `float32` the largest distance is 8.0 where the true value is 0.0144
`B`	required	`m` points as an `(m, d)` array, with the same `d` as `A`	the same loss. Centering removes it, as the section on precision shows

Cost 1: time

```
import timeit

import numpy as np


def dist_loop(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Distances between every row of A and every row of B, with two Python loops."""
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Exact. Builds an (n, m, d) temporary array."""
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Fast and small, but loses precision when points are close together and far from the
origin."""
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((300, 16)), rng.random((400, 16))
t = {f.__name__: min(timeit.repeat(lambda: f(A, B), number=1, repeat=3))
     for f in (dist_loop, dist_broadcast, dist_dot)}
assert t["dist_loop"] / t["dist_broadcast"] > 10
assert t["dist_broadcast"] / t["dist_dot"] > 2
for name, sec in t.items():
    print(f"{name:15s} {sec * 1e3:9.2f} ms")
```

```
# output varies
dist_loop          298.99 ms
dist_broadcast      13.27 ms
dist_dot             3.61 ms
```

The asserts prove two bounds. The broadcast version is at least 10 times faster than the loop. The dot version is at least 2 times faster than the broadcast version. The dot version never forms the `(n, m, d)` array, and that is a likely cause of its speed. This book did not test it.

Cost 2: memory

The broadcast version builds an array with `n * m * d` values. For 2000 points in 64 dimensions, that is 256 million values, which is 2 GB.

```
import tracemalloc

import numpy as np


def peak_mb(f, *args):
    tracemalloc.start()
    f(*args)
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return peak / 1e6


def dist_broadcast(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Exact. Builds an (n, m, d) temporary array."""
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Fast and small, but loses precision when points are close together and far from the
origin."""
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


rng = np.random.default_rng(0)
A, B = rng.random((2000, 64)), rng.random((2000, 64))
print("result alone:       ", 2000 * 2000 * 8 / 1e6, "MB")
print("broadcast peak:     ", round(peak_mb(dist_broadcast, A, B)), "MB")
print("dot peak:           ", round(peak_mb(dist_dot, A, B)), "MB")
```

```
result alone:        32.0 MB
broadcast peak:      2080 MB
dot peak:            96 MB
```

The result itself is 32 MB. The dot version peaks at 96 MB, which is 3 times the size of the result. The broadcast version peaks at 2080 MB, which is 65 times the size of the result. The temporary array grows with `n * m * d`. At 3000 points it alone has 3000 × 3000 × 64 × 8 bytes, which is 4.6 GB.

Two remedies exist for the broadcast version. You can process `A` in blocks of rows, so that the temporary array has a fixed size. Or you can use the dot version, and read the next section first.

Cost 3: precision

The dot version subtracts two large numbers that are almost equal. The squared norms `|a|²` and `|b|²` are large when the points are far from the origin. The squared distance is small when the points are close to each other. The difference loses digits.

```
import numpy as np


def dist_loop(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Distances between every row of A and every row of B, with two Python loops."""
    out = np.empty((len(A), len(B)))
    for i, a in enumerate(A):
        for j, b in enumerate(B):
            out[i, j] = np.sqrt(((a - b) ** 2).sum())
    return out


def dist_broadcast(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Exact. Builds an (n, m, d) temporary array."""
    return np.sqrt(((A[:, None, :] - B[None, :, :]) ** 2).sum(-1))


def dist_dot(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """Fast and small, but loses precision when points are close together and far from the
origin."""
    d2 = (A ** 2).sum(1)[:, None] + (B ** 2).sum(1)[None, :] - 2 * A @ B.T
    return np.sqrt(np.maximum(d2, 0))


def dist_dot_centered(A: np.ndarray, B: np.ndarray) -> np.ndarray:
    """The dot version, after moving the origin to the middle of the data."""
    c = np.concatenate([A, B]).mean(axis=0)
    return dist_dot(A - c, B - c)


rng = np.random.default_rng(0)
P = rng.random((60, 3)) * 0.01 + 1e4        # 60 points inside a cube of side 0.01, at 10,000
ref = dist_loop(P, P)
print("mean true distance:      ", round(float(ref.mean()), 4))
print("dot            max error:", f"{np.abs(dist_dot(P, P) - ref).max():.1e}")
print("dot centered   max error:", f"{np.abs(dist_dot_centered(P, P) - ref).max():.1e}")

P32 = P.astype(np.float32)
print("float32, largest distance, true:    ", round(float(dist_broadcast(P32, P32).max()), 4))
print("float32, largest distance, dot:     ", float(dist_dot(P32, P32).max()))
print("float32, largest distance, centered:", round(float(dist_dot_centered(P32, P32).max()),
4))
```

```
mean true distance:       0.0069
dot            max error: 3.5e-04
dot centered   max error: 1.2e-10
float32, largest distance, true:     0.0144
float32, largest distance, dot:      8.0
float32, largest distance, centered: 0.0144
```

The first three lines are for `float64`. The error of the plain dot version is 3.5e-4, and the mean true distance is 6.9e-3. The error is 5 percent of the answer. After centering, the error is 1.2e-10.

The `float32` lines are worse. The squared norms are about 3e8. At that size the gap between neighbor `float32` values is 32. The plain dot version reports a largest distance of 8.0. The true value is 0.0144. The answer is wrong by a

factor of more than 500.

The centered version is correct. After the origin moves to the middle of the data, the numbers are small. Then `float32` has digits to spare.

`np.concatenate`

```
np.concatenate(arrays, /, axis=0, out=None, *, dtype=None, casting='same_kind')
```

The function joins arrays along an existing axis. The function below uses it to put the two point sets in one array before it takes the mean.

parameter	default	what it does	what goes wrong
`arrays`	required	a sequence of arrays. All shapes must match, except along `axis`	other shapes raise `ValueError` and name the axis that differs
`axis`	`0`	the axis to join along	with `None`, the arrays are flattened first

`dist_dot_centered`

```
dist_dot_centered(A: numpy.ndarray, B: numpy.ndarray) -> numpy.ndarray
```

The function moves the origin to the mean of both point sets, then calls `dist_dot`.

parameter	default	what it does	what goes wrong
`A`	required	`n` points as an `(n, d)` array	the points must be in a small range. If they are spread over a large range, centering does not help
`B`	required	`m` points as an `(m, d)` array, with the same `d` as `A`	a different `d` raises `ValueError`

Centering is the fix to remember. It costs one mean and two subtractions. It does not help when the points are spread over a large range. In that case the loss is a property of the problem and not of the method.

Which one to use

Version	Time	Peak memory	Precision
`dist_loop`	slowest by far	the result	exact
`dist_broadcast`	middle	`n * m * d` values	exact
`dist_dot_centered`	fastest	a few results	good when centered

Use the dot version with centering as the default, and test it against the broadcast version on a small sample of your data. Use the broadcast version when you need the exact differences, for example when you also need the direction `a - b`. Use the loop only to check the others.

How it works

The rule is a small algorithm. Align the shapes on the right, and for each position take the one length that is not 1. If two lengths differ and neither is 1, stop with an error. The next block writes it in ten lines, and tests it against `np.broadcast_shapes` on 2000 random pairs of shapes.

```
import numpy as np


def broadcast_shape(*shapes):
    """The broadcast rule written out: right-aligned, equal or 1."""
    n = max(len(s) for s in shapes)
    padded = [(1,) * (n - len(s)) + tuple(s) for s in shapes]
    out = []
    for lengths in zip(*padded):
        not_one = {length for length in lengths if length != 1}
        if len(not_one) > 1:
            raise ValueError(f"lengths {sorted(not_one)} cannot broadcast")
        out.append(not_one.pop() if not_one else 1)
    return tuple(out)


rng = np.random.default_rng(0)
agree = valid = 0
for _ in range(2000):
    shapes = [tuple(int(x) for x in rng.choice([1, 2, 3], size=rng.integers(0, 4))) for _ in
range(2)]
    try:
        want = np.broadcast_shapes(*shapes)
    except ValueError:
        want = None
    try:
        got = broadcast_shape(*shapes)
    except ValueError:
        got = None
    agree += got == want
    valid += want is not None

print("pairs tested:       ", 2000)
print("pairs that broadcast:", valid)
print("pairs that agree:    ", agree)
```

```
pairs tested:        2000
pairs that broadcast: 1649
pairs that agree:     2000
```

The loops over the stretched axes cost nothing in memory, because the stride is 0. The next block shows the strides that the rule gives, and then follows the shapes through one call of `dist_broadcast`.

```
import numpy as np

col, row = np.broadcast_arrays(np.arange(3.0)[:, None], np.arange(4.0))
print("shapes: ", col.shape, row.shape)
print("strides:", col.strides, row.strides)

rng = np.random.default_rng(0)
A, B = rng.random((5, 2)), rng.random((7, 2))
steps = [
    ("A[:, None, :]", A[:, None, :]),
    ("B[None, :, :]", B[None, :, :]),
    ("difference", A[:, None, :] - B[None, :, :]),
    ("squared and summed over d", ((A[:, None, :] - B[None, :, :]) ** 2).sum(-1)),
]
for name, value in steps:
    print(f"{name:28s} {value.shape}")
```

```
shapes:  (3, 4) (3, 4)
strides: (8, 0) (0, 8)
A[:, None, :]                (5, 1, 2)
B[None, :, :]                (1, 7, 2)
difference                   (5, 7, 2)
squared and summed over d    (5, 7)
```

Each stretched axis has stride 0, so the two inputs are read in place and the subtraction makes the one new array, `(5, 7, 2)`. This array is the cost. It has `n * m * d` values, and a sum over the last axis then shrinks it to the `(5, 7)` result.

The invariant. The shape of the result is the right-aligned maximum of the input shapes, and the rule makes no copy of the inputs. The block above tests the rule against NumPy on 2000 cases.

The failure. A shape that is almost right passes. A `(n,)` against an `(n, 1)` is allowed, and the result is `(n, n)`. The error comes only when two lengths differ and neither is 1. Test the shape of a result, because the rule cannot know what you meant.

Reference

Callable	Introduced in	Contract
`np.broadcast_shapes`	The rule	the result shape for given shapes, with no array
`np.broadcast_to`	A stretch costs no memory	a read-only view with the target shape
`np.mean`	The trap: a result you did not mean	mean along an axis, with `keepdims`
`dist_loop`	The function you build	distances with two Python loops
`dist_broadcast`	The function you build	distances with one broadcast subtraction
`dist_dot`	The function you build	distances from norms and a matrix product
`np.concatenate`	Cost 3: precision	join arrays along an existing axis
`dist_dot_centered`	Cost 3: precision	the dot version, with the origin in the middle

What you have now

You have the broadcasting rule and `np.broadcast_shapes` to test it. You know that a stretch is a stride of 0 and costs nothing until a result is made. You have three distance functions, and for each one you have a measured time, a measured peak memory and a measured error.

The lesson of the chapter is that the fastest version of an array program is also the one that loses the most precision. Measure all three costs before you choose.

Chapter 6 turns to the other half of a vectorized program: the ufunc, its `reduce` and `accumulate` methods, and the cases where no vectorized form exists.

Chapter 6 — Ufuncs and vectorization

> **Build** — a moving average in four versions, a loop, a cumulative-sum version, a window version and a convolution, with a measured time and a measured precision for each.
>
> **Theory kept** — a ufunc is a function that runs element by element in compiled code. It has methods (`reduce` , `accumulate` , `outer`) that apply it along an axis. Vectorization is the act of rewriting a loop as calls to ufuncs. It does not exist for every loop.
>
> **Source** — New. The ufunc methods are in the NumPy documentation. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch06_ufuncs` , which is `examples/ch06_ufuncs.py` .

A ufunc, short for universal function, is the unit of speed in NumPy. `np.add` , `np.multiply` , `np.sqrt` , `np.maximum` and 102 others are ufuncs. The section How it works counts them. The operators `+` and `*` call them. A ufunc takes arrays, loops over the elements in compiled code, and returns an array. It does the Python-level work once for the whole array, and not once per element.

This chapter shows three methods that make a ufunc more than an element-wise function. It shows one wrong way to get the speed. It ends with a moving average that you build four times.

Three methods on every binary ufunc

A binary ufunc has a `reduce` method, an `accumulate` method and an `outer` method. Each one applies the ufunc in a different pattern.

```
import numpy as np

x = np.array([3, 1, 4, 1, 5, 9, 2, 6])
print("reduce     :", np.add.reduce(x))
print("accumulate :", np.add.accumulate(x))
print("running max:", np.maximum.accumulate(x))
print("outer      :")
print(np.multiply.outer(x[:3], x[:3]))
```

```
reduce     : 31
accumulate : [ 3  4  8  9 14 23 25 31]
running max: [3 3 4 4 5 9 9 9]
outer      :
[[ 9  3 12]
 [ 3  1  4]
 [12  4 16]]
```

`np.add`

```
np.add(x1, x2, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None,
subok=True, signature=None)
```

The ufunc adds two arrays element by element, with the broadcast rule of chapter 5. The operator `+` calls it.

parameter	default	what it does	what goes wrong
`x1`, `x2`	required	the two inputs. They broadcast against each other	a shape that does not broadcast raises `ValueError`
`out`	`None`	an array to write the result into	an `out` of a type that cannot hold the result raises `UFuncTypeError`: `float64` into `int64` is refused
`where`	`True`	a boolean array. Elements where it is `False` are skipped	without `out`, the skipped elements have undefined values
`casting`	`'same_kind'`	which casts the ufunc may do to match types	`'unsafe'` allows `float64` into `int64`, and it truncates
`dtype`	`None`	the type of the computation and of the result	`dtype=np.int64` on two `int8` values 127 and 1 gives 128 and not -128

`np.add.reduce`

```
np.add.reduce(array, /, axis=0, dtype=None, out=None, **kwargs)
```

The method folds the array along one axis to one value for each position of the other axes.

parameter	default	what it does	what goes wrong
`array`	required	the array to fold	an empty array gives the identity of the ufunc: 0 for `np.add`
`axis`	`0`	the axis to fold along. With `None`, all axes	the default is 0, and `np.sum` has `None`. On a 2-D array the two give `[3 5 7]` and `15`
`dtype`	`None`	the type of the running total	—
`out`	`None`	an array to write the result into	—

`np.add.accumulate`

```
np.add.accumulate(array, /, axis=0, dtype=None, out=None)
```

The method returns every partial result of the fold, in order.

parameter	default	what it does	what goes wrong
`array`	required	the array to scan	—
`axis`	`0`	the axis to scan along	the default is 0, and `np.cumsum` flattens with `None`
`dtype`	`None`	the type of the running total and of the result	a `float32` input keeps a `float32` total, and the drift of chapter 2 appears
`out`	`None`	an array to write the result into	—

`np.add.outer`

```
np.add.outer(A, B, /, **kwargs)
```

The method applies the ufunc to every pair of one element of `A` and one element of `B`. The result has the shape of `A` followed by the shape of `B`.

parameter	default	what it does	what goes wrong
`A`, `B`	required	the two inputs	the result has `A.ndim + B.ndim` axes, and a large pair makes a large array

The call `np.add.reduce(x)` is `x.sum()`. It folds the array to one value. The call `np.add.accumulate(x)` is `np.cumsum(x)`. It keeps every partial result. The call `np.maximum.accumulate(x)` gives the running maximum, which has no special name in NumPy. The call `np.multiply.outer` gives every product of a pair, which is the broadcast of chapter 5 written as a method.

The methods let you build many functions from the 106 ufuncs. A running minimum, a running product and a running logical `or` are one call each.

axis, keepdims, out and where

Every reduction takes `axis`, which can be an integer or a tuple. It takes `keepdims`, which chapter 5 used to keep the reduced axis with length 1. Every ufunc takes `out`, to write the result into an array that you made, and `where`, to skip some elements.

```
import numpy as np

m = np.arange(6).reshape(2, 3)
print("axis 0      :", m.sum(axis=0))
print("axis 1      :", m.sum(axis=1))
print("axes (0, 1) :", m.sum(axis=(0, 1)))
print("keepdims    :", m.sum(axis=1, keepdims=True).shape)

y = np.array([1.0, -1.0, 4.0])
out = np.full(3, np.nan)
np.sqrt(y, out=out, where=y >= 0)
print("sqrt with where:", out)
```

```
axis 0      : [3 5 7]
axis 1      : [ 3 12]
axes (0, 1) : 15
keepdims    : (2, 1)
sqrt with where: [ 1. nan  2.]
```

The last line shows the contract of `where`. The element that the condition skipped keeps its old value in `out`. If you do not pass `out`, the values of the skipped elements are undefined, so always pass it.

One more behavior matters for data with missing values. A `nan` spreads through a sum.

```
import numpy as np

x = np.array([1.0, np.nan, 2.0])
print("sum   :", x.sum())
print("nansum:", np.nansum(x))
print("mean  :", x.mean(), "nanmean:", np.nanmean(x))
```

```
sum   : nan
nansum: 3.0
mean  : nan nanmean: 1.5
```

`np.nansum`

```
np.nansum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=
<no value>)
```

The function adds the values of an array and treats `nan` as 0. It takes the same arguments as `np.sum`.

parameter	default	what it does	what goes wrong
`a`	required	the array	an array of only `nan` gives `0.0`, which looks like a real sum
`axis`	`None`	the axis to add along	—
`dtype`	`None`	the type of the running total	—

The `nan` functions skip the missing values. Use them with intent. A silent `nansum` on a column that is mostly `nan` gives a number, and the number is a sum of very few values.

Temporaries and `out=`

An expression such as `a * b + c` makes a temporary array for `a * b`, and then a second array for the sum. For small arrays this does not matter. For large arrays, each temporary is a pass over memory and an allocation. Writing into a result array that you made in advance avoids both.

```
import timeit

import numpy as np

rng = np.random.default_rng(1)
a = rng.random(10_000_000)
b = rng.random(10_000_000)
c = rng.random(10_000_000)
out = np.empty_like(a)


def expression():
    return a * b + c


def in_place():
    np.multiply(a, b, out=out)
    np.add(out, c, out=out)
    return out


assert np.array_equal(expression(), in_place())
t_expr = min(timeit.repeat(expression, number=1, repeat=5))
t_inpl = min(timeit.repeat(in_place, number=1, repeat=5))
assert t_expr / t_inpl > 1.1
print(f"expression / in place: {t_expr / t_inpl:.2f}x")
```

```
# output varies
expression / in place: 1.47x
```

The assert requires a gain of at least 1.1 times. This is a modest gain, and the code is harder to read. Use `out=` when the profile shows that this line matters, and not before.

`np.vectorize` is not vectorization

The name suggests speed. The documentation says otherwise: the function is provided for convenience, and it is a Python loop. Measure it against a plain loop and against the real vectorized form.

```
import timeit

import numpy as np

v = np.random.default_rng(0).random(1_000_000)


def f(a):
    return a * a + 1 if a > 0.5 else a


vf = np.vectorize(f)
t_loop = min(timeit.repeat(lambda: [f(a) for a in v.tolist()], number=1, repeat=3))
t_vec = min(timeit.repeat(lambda: vf(v), number=1, repeat=3))
t_where = min(timeit.repeat(lambda: np.where(v > 0.5, v * v + 1, v), number=1, repeat=3))

assert t_vec / t_loop > 0.5          # np.vectorize is no faster than the loop
assert t_loop / t_where > 5          # the real vectorized form is
print(f"list comprehension {t_loop * 1e3:7.1f} ms")
print(f"np.vectorize       {t_vec * 1e3:7.1f} ms")
print(f"np.where           {t_where * 1e3:7.1f} ms")
```

```
# output varies
list comprehension   183.9 ms
np.vectorize         227.1 ms
np.where              15.1 ms
```

`np.vectorize`

```
np.vectorize(pyfunc=<no value>, otypes=None, doc=None, excluded=None, cache=False,
signature=None)
```

The class wraps a Python function so that it accepts arrays. It calls the function once for each element.

parameter	default	what it does	what goes wrong
`pyfunc`	`<no value>`	the Python function to wrap	it is called in a Python loop, so the speed is the speed of a loop
`otypes`	`None`	the output types. Without it, NumPy takes the type of the first result	`0 if v < 1 else v` on `[0.5, 2.7]` gives `[0 2]`, because the first result is an integer
`signature`	`None`	a generalized signature such as `(n),(n)->()` for a function of arrays	a wrong signature raises `ValueError` at the call

In the run above, the `np.vectorize` call was slower than the list comprehension that it replaces. The assert proves only that it is not more than 2 times faster. The assert also proves that the real vectorized form, with `np.where`, is more than 5 times faster than the loop. Use `np.vectorize` to make a Python function accept arrays in a place where speed does not matter. Do not use it to go faster.

The function you build: a moving average

A moving average of window `w` replaces each value with the mean of the `w` values that end there. The loop version is the definition. Three others follow.

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_loop(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average of window w, from the definition."""
    out = np.empty(len(x) - w + 1)
    for i in range(len(out)):
        out[i] = x[i:i + w].mean()
    return out


def ma_cumsum(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a cumulative sum."""
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_window(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a sliding-window view."""
    return sliding_window_view(x, w).mean(axis=-1)


def ma_convolve(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average as a convolution with a flat kernel."""
    return np.convolve(x, np.ones(w) / w, mode="valid")


x = np.random.default_rng(2).random(20_000)
ref = ma_loop(x, 50)
for f in (ma_cumsum, ma_window, ma_convolve):
    print(f"{f.__name__:11s} max difference from the loop: {np.abs(f(x, 50) -
ref).max():.1e}")
```

```
ma_cumsum   max difference from the loop: 2.4e-13
ma_window   max difference from the loop: 0.0e+00
ma_convolve max difference from the loop: 3.3e-16
```

All three agree with the loop to within rounding. The version `ma_cumsum` is based on one fact: the sum of the window that ends at `i` is the cumulative sum at `i` minus the cumulative sum `w` steps earlier. This is two passes over the data, whatever the size of `w`. The version `ma_window` uses `sliding_window_view`, which makes an `(n - w + 1, w)` view without copying, as chapter 3 explained, and then takes the mean of each row. The version `ma_convolve` treats the average as a convolution with a flat kernel.

`np.lib.stride_tricks.sliding_window_view`

```
np.lib.stride_tricks.sliding_window_view(x, window_shape, axis=None, *, subok=False,
writeable=False)
```

The function returns a read-only view in which each row is one window of the input. It copies nothing.

parameter	default	what it does	what goes wrong
`x`	required	the array	—
`window_shape`	required	the length of the window, or a tuple of lengths	a window longer than the input raises `ValueError`
`axis`	`None`	the axis or axes to slide along	with `None`, the window slides over all axes, and `window_shape` must have one length for each
`writeable`	`False`	with `True`, writes are allowed	a write changes every window that holds that position

`np.convolve`

```
np.convolve(a, v, mode='full')
```

The function returns the discrete convolution of two 1-D arrays. With a flat kernel, it is a moving sum.

parameter	default	what it does	what goes wrong
`a`, `v`	required	the signal and the kernel	when `v` is longer than `a`, NumPy swaps them, and `ma_convolve` returns 3 values for 3 inputs and `w = 5`
`mode`	`'full'`	`'full'` returns `n + w - 1` values. `'valid'` returns `n - w + 1` values, only where the kernel fits	`'full'` has partial windows at both ends, and they are not averages of `w` values

The next five functions have the same two parameters. Each section below gives what goes wrong for that version.

`ma_loop`

```
ma_loop(x: numpy.ndarray, w: int) -> numpy.ndarray
```

The function computes the moving average from the definition, one window at a time.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of values	the slowest version in the timing block, scaled from 20,000 values
`w`	required	the window length, a positive integer	a `w` larger than the input raises `ValueError`

`ma_cumsum`

```
ma_cumsum(x: numpy.ndarray, w: int) -> numpy.ndarray
```

The function computes the moving average from a cumulative sum.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of values	`float32` values from 10,000 to 10,100 give an error of 207
`w`	required	the window length, a positive integer	a `w` larger than the input gives an empty array and no error

`ma_cumsum_centered`

```
ma_cumsum_centered(x: numpy.ndarray, w: int) -> numpy.ndarray
```

The function subtracts the mean first, so the cumulative sum stays small, and adds it back after.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of values	the error is 0 on seed 2 and 4.7e-4 on seed 3, for `float32` values from 10,000 to 10,100
`w`	required	the window length, a positive integer	the same empty result as `ma_cumsum` for a `w` larger than the input

`ma_window`

```
ma_window(x: numpy.ndarray, w: int) -> numpy.ndarray
```

The function averages each row of a sliding-window view.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of values	the slowest of the three fast versions in the timing block
`w`	required	the window length, a positive integer	a `w` larger than the input raises `ValueError`

`ma_convolve`

```
ma_convolve(x: numpy.ndarray, w: int) -> numpy.ndarray
```

The function averages with a convolution and a flat kernel.

parameter	default	what it does	what goes wrong
`x`	required	a 1-D array of values	—
`w`	required	the window length, a positive integer	a `w` larger than the input returns `len(x)` values that are not moving averages

Time them on one million values with `w = 50`.

```
import timeit

import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_loop(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average of window w, from the definition."""
    out = np.empty(len(x) - w + 1)
    for i in range(len(out)):
        out[i] = x[i:i + w].mean()
    return out


def ma_cumsum(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a cumulative sum."""
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_window(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a sliding-window view."""
    return sliding_window_view(x, w).mean(axis=-1)


def ma_convolve(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average as a convolution with a flat kernel."""
    return np.convolve(x, np.ones(w) / w, mode="valid")


x = np.random.default_rng(2).random(1_000_000)
t_loop = min(timeit.repeat(lambda: ma_loop(x[:20_000], 50), number=1, repeat=3)) * 50   #
scaled to 1,000,000
t = {f.__name__: min(timeit.repeat(lambda: f(x, 50), number=1, repeat=3))
     for f in (ma_cumsum, ma_window, ma_convolve)}
assert t_loop / t["ma_cumsum"] > 20
print(f"ma_loop (scaled from 20,000 values) {t_loop * 1e3:8.0f} ms")
for name, sec in t.items():
    print(f"{name:35s} {sec * 1e3:8.0f} ms")
```

```
# output varies
ma_loop (scaled from 20,000 values)     2673 ms
ma_cumsum                                 14 ms
ma_window                                 52 ms
ma_convolve                               18 ms
```

The loop is measured on 20,000 values and multiplied by 50, so that the run is short. The assert proves that the cumulative-sum version is more than 20 times faster. In the run above, the window version was the slowest of the three. It computes a real mean over a `w`-wide row for every output, which is a likely cause. This book did not test it. The cumulative-sum version and the convolution were within a factor of 2 of each other.

The precision trap in the fast version

The cumulative-sum version is the fastest and the shortest. It is also the one that chapter 2 warned about. Its cumulative sum grows large, and every window sum is the difference of two large numbers.

```
import numpy as np
from numpy.lib.stride_tricks import sliding_window_view


def ma_cumsum(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a cumulative sum."""
    c = np.concatenate([[0.0], np.cumsum(x)])
    return (c[w:] - c[:-w]) / w


def ma_cumsum_centered(x: np.ndarray, w: int) -> np.ndarray:
    m = x.mean()
    return ma_cumsum(x - m, w) + m


def ma_window(x: np.ndarray, w: int) -> np.ndarray:
    """Moving average from a sliding-window view."""
    return sliding_window_view(x, w).mean(axis=-1)


print("float32 data, values from 10000 to 10100, one million values")
for seed in (2, 3):
    s = np.random.default_rng(seed).random(1_000_000)
    x32 = (s * 100 + 1e4).astype(np.float32)
    truth = ma_window(x32.astype(np.float64), 50)
    for f in (ma_cumsum, ma_cumsum_centered, ma_window):
        print(f"seed {seed} {f.__name__:20s} max error {np.abs(f(x32, 50) -
truth).max():.3g}")
    centered_total = np.abs(np.cumsum(x32 - x32.mean())).max()
    print(f"seed {seed} largest centered running total {float(centered_total):.0f} (2**14 is
16384), gap at 10050: {float(np.spacing(np.float32(10050))):.6f}")
    x64 = x32.astype(np.float64)
    print(f"seed {seed} {'ma_cumsum in float64':20s} max error {np.abs(ma_cumsum(x64, 50) -
truth).max():.3g}")
```

```
float32 data, values from 10000 to 10100, one million values
seed 2 ma_cumsum            max error 207
seed 2 ma_cumsum_centered   max error 0
seed 2 ma_window            max error 0.00197
seed 2 largest centered running total 13611 (2**14 is 16384), gap at 10050: 0.000977
seed 2 ma_cumsum in float64 max error 0
seed 3 ma_cumsum            max error 209
seed 3 ma_cumsum_centered   max error 0.000469
seed 3 ma_window            max error 0.00186
seed 3 largest centered running total 30083 (2**14 is 16384), gap at 10050: 0.000977
seed 3 ma_cumsum in float64 max error 0
```

The plain cumulative-sum version has an error of about 200 on values near 10,000. That is 2 percent of the values. The window version has an error of 0.002, which is about twice the spacing of `float32` at that size.

The centered version subtracts the mean first, so the cumulative sum stays small. On seed 2 its error is exactly 0. This is a property of that data and not of the method.

Every value is a multiple of 2^{-10}, which is the gap at 10,050. The `float32` format holds a multiple of 2^{-10} exactly up to 2^{14}, which is 16,384. The block prints the largest centered total: 13,611 on seed 2, below that limit, and 30,083 on seed 3, above it.

On seed 3 the error is 4.7e-4, which is smaller than the window version.

Read the two seeds together. Centering gives a good answer in both cases, and the exact zero on one seed is luck.

If you do not know the range of your data, use the window version or use `float64` for the cumulative sum.

The last line of each seed shows the `float64` result. Its error is 0 on both seeds, for the same reason as the exact zero above. The values sit on a grid of 2^{-10}, and `float64` holds their sums exactly.

When no vectorized form exists

Some loops cannot be written with ufuncs. The usual cause is a dependency: each value needs the result for the previous value. An exponential moving average is the standard case.

```
import time

import numpy as np


def ema_loop(x, alpha):
    out = np.empty_like(x)
    acc = x[0]
    for i, v in enumerate(x.tolist()):
        acc = alpha * v + (1 - alpha) * acc
        out[i] = acc
    return out


x = np.random.default_rng(2).random(1_000_000)
start = time.perf_counter()
y = ema_loop(x, 0.1)
elapsed = time.perf_counter() - start
assert np.isfinite(y[-1]) and elapsed < 10
print(f"{len(x) / elapsed / 1e6:.1f} million steps a second")
```

```
# output varies
3.1 million steps a second
```

The loop above ran at the rate that the block printed. The plain ufunc methods do not apply to this loop directly, because each step needs the result of the step before it.

A rewrite with cumulative sums and powers of `1 - alpha` exists, and this book did not test its numerical behavior. Two honest options exist outside this book. A filter routine from a signal library applies this recurrence in compiled

code. A compiler such as Numba turns the loop into machine code. This book did not measure either, so it makes no claim about their speed.

How it works

A ufunc is a table of compiled loops, one for each combination of input types. When you call it, NumPy finds the first loop in the table to which both inputs can be cast safely. It then runs that loop over the whole array. (The NumPy documentation describes this rule. This book did not measure the buffers that the cast uses.) The type letters are the names of the dtype: `b` is `int8`, `f` is `float32`, `d` is `float64`.

```
import numpy as np

ufuncs = [name for name in dir(np) if isinstance(getattr(np, name), np.ufunc)]
print("ufuncs in the numpy namespace:", len(ufuncs))
print("compiled loops of np.add:     ", len(np.add.types))
print("first six loops:              ", np.add.types[:6])
for a, b in [(np.int8, np.int8), (np.int8, np.float32), (np.int64, np.uint64)]:
    print(f"{a.__name__:7s} + {b.__name__:8s} runs in", np.result_type(a, b))
```

```
ufuncs in the numpy namespace: 106
compiled loops of np.add:      22
first six loops:               ['??->?', 'bb->b', 'BB->B', 'hh->h', 'HH->H', 'ii->i']
int8    + int8     runs in int8
int8    + float32  runs in float32
int64   + uint64   runs in float64
```

The three results at the end are the promotion rules of chapter 2, read from the same table. The pair `int64` and `uint64` has no integer loop that holds both, so it runs as `float64`.

The two methods are also simple. `reduce` is a fold, and `accumulate` is a scan. The next block writes both with plain Python and compares them with NumPy.

```
import functools
import itertools
import operator

import numpy as np

x = np.random.default_rng(0).random(1000)

fold = functools.reduce(operator.add, x.tolist())
scan = list(itertools.accumulate(x.tolist()))

print("reduce equals the fold:        ", np.isclose(np.add.reduce(x), fold))
print("reduce equals it exactly:      ", np.add.reduce(x) == fold)
print("difference:                    ", abs(np.add.reduce(x) - fold))
print("accumulate equals the scan:    ", np.allclose(np.add.accumulate(x), scan))
print("identity of add and multiply:  ", np.add.identity, np.multiply.identity)
print("reduce of an empty array, add: ", np.add.reduce([]))
try:
    np.maximum.reduce([])
except ValueError as e:
    print("reduce of an empty array, max: ValueError -", e)
```

```
reduce equals the fold:         True
reduce equals it exactly:       False
difference:                     1.1368683772161603e-13
accumulate equals the scan:     True
identity of add and multiply:   0 1
reduce of an empty array, add:  0.0
reduce of an empty array, max: ValueError - zero-size array to reduction operation maximum
which has no identity
```

The fold and the NumPy result agree to rounding but not exactly. `np.add.reduce` adds in pairs, as chapter 2 explained, and the Python fold adds one value at a time. The difference is of the order of 1e-13 for 1000 values.

The identity is the value of a fold over no elements: 0 for addition and 1 for multiplication. The ufunc `np.maximum` has none, so an empty reduction is an error and not a wrong number.

The invariant. For an associative ufunc, `reduce` gives the fold of the operator. The method `accumulate` gives every partial result. Both hold up to the order of the additions.

The failure. A float fold depends on the order of the additions. A Python loop and a NumPy call give different last digits. A running total in a narrow type drifts, as in chapter 2. Compare results with `np.isclose` and not with `==`.

Reference

Callable	Introduced in	Contract
`np.add`	Three methods on every binary ufunc	element-wise sum, with the broadcast rule
`np.add.reduce`	Three methods on every binary ufunc	fold along an axis
`np.add.accumulate`	Three methods on every binary ufunc	every partial result of the fold
`np.add.outer`	Three methods on every binary ufunc	the operator on every pair
`np.nansum`	axis, keepdims, out and where	sum that skips `nan`
`np.vectorize`	`np.vectorize` is not vectorization	a Python function applied to each element
`np.lib.stride_tricks.sliding_window_view`	The function you build: a moving average	read-only view of all windows
`np.convolve`	The function you build: a moving average	discrete convolution of two 1-D arrays
`ma_loop`	The function you build: a moving average	moving average from the definition
`ma_cumsum`	The function you build: a moving average	moving average from a cumulative sum
`ma_cumsum_centered`	The precision trap in the fast version	the cumulative-sum version after centering
`ma_window`	The function you build: a moving average	moving average from a window view
`ma_convolve`	The function you build: a moving average	moving average from a convolution

What you have now

You have the three ufunc methods and four arguments: `axis`, `keepdims`, `out` and `where`. You have a moving average in four versions, with a time and an error for each. You know that `np.vectorize` is a loop. You know that `out=` saves a factor of 1.4 on large arrays. You know that the cumulative-sum trick needs centered data in `float32`.

The test that you can apply to any loop is short. If each iteration needs only its own element and its neighbors at fixed offsets, a vectorized form exists. If an iteration needs the result of the previous one, it probably does not.

Chapter 7 uses NumPy for the two jobs that most users meet next: solving linear systems and drawing random numbers that you can reproduce.

Chapter 7 — Linear algebra and random numbers

> **Build** — `fit_line(x, y)`, a least-squares line with a bootstrap interval for the slope, which gives the same answer every time that you give it the same seed.
>
> **Theory kept** — solve a system with `solve` and not with the inverse. Fit with `lstsq` and not with the normal equations. Make random numbers from a `Generator` that you create and pass, and not from a global state.
>
> **Source** — New. The conditioning rule is standard numerical linear algebra. The random-number policy is from the NumPy documentation. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch07_linalg_random`, which is `examples/ch07_linalg_random.py`.

Two jobs account for most of the NumPy code outside data cleaning. One is to solve for unknown numbers, such as the coefficients of a fit. The other is to draw random numbers, for a simulation, a split or a resample. Both have a textbook form that works on small examples and fails on real ones. This chapter measures how each one fails.

The matrix product

The operator `@` is the matrix product. It works on vectors, matrices and stacks of matrices.

```
import numpy as np

A = np.arange(6.0).reshape(2, 3)
v = np.array([1.0, 0.0, -1.0])
stack = np.arange(24.0).reshape(4, 2, 3)

print("matrix @ vector      :", (A @ v).shape)
print("stack of 4 @ vector  :", (stack @ v).shape)
print("stack of 4 @ A.T     :", (stack @ A.T).shape)
```

```
matrix @ vector      : (2,)
stack of 4 @ vector  : (4, 2)
stack of 4 @ A.T     : (4, 2, 2)
```

The last two lines show the rule. Leading axes act as a stack, and `@` multiplies the last two axes of each matrix. The leading axes broadcast, as in chapter 5. One call does four matrix products.

`np.matmul`

```
np.matmul(x1, x2, /, out=None, *, axes=<no value>, axis=<no value>, keepdims=False,
casting='same_kind', order='K', dtype=None, subok=True, signature=None)
```

The ufunc computes the matrix product. The operator `@` calls it.

parameter	default	what it does	what goes wrong
`x1`, `x2`	required	the two inputs. The last axis of `x1` must match the second-to-last axis of `x2`. Leading axes broadcast	a mismatch raises `ValueError`. The `*` operator gives the right shape and the wrong values for two square matrices
`out`	`None`	an array to write the result into	—
`dtype`	`None`	the type of the computation and of the result	—

Use `@` and `np.dot` only on arrays. Do not use the `*` operator for a matrix product. It multiplies element by element, and for two square matrices it gives a result with the right shape and the wrong values.

Solve, do not invert

To solve `A x = b` you can compute `inv(A) @ b` or call `solve(A, b)`. Both give the same answer in a textbook. Test them on a Hilbert matrix, which is a standard badly conditioned matrix. The right answer is a vector of ones, because the code builds `b` from it.

```
import numpy as np


def hilbert(n: int) -> np.ndarray:
    i = np.arange(n)
    return 1.0 / (i[:, None] + i[None, :] + 1)


print(f"{'n':>3} {'cond(A)':>9} {'solve error':>12} {'inv error':>10} {'solve resid':>12}
{'inv resid':>10}")
for n in (4, 8, 12):
    H = hilbert(n)
    true = np.ones(n)
    b = H @ true
    x_solve = np.linalg.solve(H, b)
    x_inv = np.linalg.inv(H) @ b
    print(f"{n:>3} {np.linalg.cond(H):9.1e} {np.abs(x_solve - true).max():12.1e} "
          f"{np.abs(x_inv - true).max():10.1e} {np.abs(H @ x_solve - b).max():12.1e} "
          f"{np.abs(H @ x_inv - b).max():10.1e}")
```

```
  n   cond(A)  solve error  inv error  solve resid  inv resid
  4   1.6e+04      6.6e-14    9.1e-13      0.0e+00    9.5e-14
  8   1.5e+10      1.2e-07    1.9e-06      2.2e-16    1.4e-08
 12   1.8e+16      7.0e-01    2.1e+01      4.4e-16    5.4e-02
```

`np.linalg.solve`

```
np.linalg.solve(a, b)
```

The function solves `a @ x = b` for `x` by a factorization of `a`. It does not form the inverse.

parameter	default	what it does	what goes wrong
`a`	required	a square matrix, or a stack of square matrices	a singular matrix raises `LinAlgError: Singular matrix`. A nearly singular matrix gives no error and a wrong answer
`b`	required	the right side, with shape `(n,)` or `(n, k)`	the residual is small even when `x` is wrong, so a small residual does not prove a small error

`np.linalg.cond`

```
np.linalg.cond(x, p=None)
```

The function returns the condition number of a matrix: the factor by which a small change in the input can grow in the answer.

parameter	default	what it does	what goes wrong
`x`	required	the matrix	for a singular matrix the result is a very large number or `inf`
`p`	`None`	the norm. With `None`, the 2-norm, from the singular values	other norms cost less and give other values

Read the table in three parts.

The condition number `cond(A)` measures how much the answer can change when the input changes a little. A rule of thumb is that the error of the answer is about `cond(A)` times the rounding unit of `float64`, which is 1.1e-16. For `n = 8`, that is 1.5e10 times 1.1e-16, or about 2e-6. The table shows 1.2e-7 for `solve`, and 1.9e-6 for `inv`. The rule gives the order of magnitude.

For `n = 12` the condition number is 1.8e16, which is more than the number of digits that `float64` has. No method can give a good answer. Both are wrong, and the errors are 0.7 and 21.

The last two columns show the real difference. The residual `H @ x - b` of `solve` is at the rounding level for every `n`, even when `x` is wrong. The residual of `inv` is 6e7 times larger at `n = 8` and 1e14 times larger at `n = 12`. The inverse method gives an answer that does not even satisfy the equation. Use the residual as a check: it tells you if the method did its job, where the error tells you only that the problem was hard.

Speed points the same way. The next block times both on a 1000-by-1000 system. It pins the linear algebra library to one thread before it imports NumPy. A shared machine then gives the same ratio from run to run.

```
import os

os.environ["OPENBLAS_NUM_THREADS"] = "1"
os.environ["OMP_NUM_THREADS"] = "1"

import timeit

import numpy as np

rng = np.random.default_rng(0)
A, b = rng.random((1000, 1000)), rng.random(1000)

t_solve = min(timeit.repeat(lambda: np.linalg.solve(A, b), number=1, repeat=5))
t_inv = min(timeit.repeat(lambda: np.linalg.inv(A) @ b, number=1, repeat=5))
assert t_inv / t_solve > 1.5
print(f"solve        {t_solve:.3f} s")
print(f"inv(A) @ b   {t_inv:.3f} s")
print(f"ratio        {t_inv / t_solve:.1f}x")
```

```
# output varies
solve        0.038 s
inv(A) @ b   0.133 s
ratio        3.5x
```

The inverse was slower and gave a worse answer. The assert requires a ratio above 1.5 with one thread.

Fit with `lstsq`, not with the normal equations

A least-squares fit minimizes `|X β − y|²`. The textbook solution is the normal equations, `(XᵀX) β = Xᵀy`. The problem is that `XᵀX` has the square of the condition number of `X`. Compare the two on polynomial fits, where `y` is built from known coefficients of 1.

```
import numpy as np

x = np.linspace(0, 1, 50)
print(f"{'degree':>6} {'cond(X)':>9} {'cond(XtX)':>10} {'lstsq error':>12} {'normal-eq
error':>16}")
for degree in (3, 6, 9, 12):
    X = np.vander(x, degree + 1)
    y = X @ np.ones(degree + 1)
    via_lstsq = np.linalg.lstsq(X, y, rcond=None)[0]
    via_normal = np.linalg.solve(X.T @ X, X.T @ y)
    print(f"{degree:>6} {np.linalg.cond(X):9.1e} {np.linalg.cond(X.T @ X):10.1e}
{np.abs(via_lstsq - 1).max():12.1e} "
          f"{np.abs(via_normal - 1).max():16.1e}")
```

```
degree   cond(X)  cond(XtX)  lstsq error  normal-eq error
     3   1.2e+02    1.4e+04      7.1e-15          1.4e-12
     6   2.0e+04    3.9e+08      8.1e-13          4.1e-08
     9   3.6e+06    1.3e+13      5.4e-11          1.3e-03
    12   6.8e+08    4.7e+17      9.7e-09          3.9e-01
```

`np.linalg.lstsq`

```
np.linalg.lstsq(a, b, rcond=None)
```

The function returns the vector `beta` that minimizes the sum of squares of `a @ beta - b`. It returns a tuple of four values: the solution, the residuals, the rank and the singular values.

parameter	default	what it does	what goes wrong
`a`	required	the design matrix, with one row for each observation	two equal columns give a rank below the number of columns, and the coefficients are not unique
`b`	required	the observations, with shape `(m,)` or `(m, k)`	the residuals are empty when the rank is below the number of columns
`rcond`	`None`	singular values below `rcond` times the largest are treated as zero. With `None`, the machine precision times the larger side	a larger `rcond` drops small directions and can change the fit

At degree 9 the error of the normal equations is 1.3e-3 and the error of `lstsq` is 5.4e-11. The normal equations lose 7 digits more. At degree 12 the error is 0.39 where `lstsq` has 1e-8.

The likely cause is the squared condition number. The column `cond(XtX)` shows that $X^\intercal X$ has about the square of the condition number of `X`: 6.8e8 becomes 4.7e17 at degree 12. The inverse of the `float64` epsilon is 4.5e15, so that value is past the limit of `float64`. The function `lstsq` uses a factorization of `X` that does not form $X^\intercal X$.

The function also returns the rank and the singular values, and that is its second use. If the rank is less than the number of columns, two of your columns carry the same information, and the coefficients are not unique. Check the rank of any design matrix that you did not build yourself.

Random numbers: create a generator and pass it

NumPy has two random-number interfaces. The old one is the global `np.random.seed` and `np.random.rand`. The current one is `np.random.default_rng`, which returns a `Generator` object. Use the current one.

```
import numpy as np

a = np.random.default_rng(42).random(3)
b = np.random.default_rng(42).random(3)
c = np.random.default_rng(43).random(3)
print("seed 42:", a)
print("seed 42:", b)
print("seed 43:", c)
print("same seed, same numbers:", np.array_equal(a, b))
```

```
seed 42: [0.77395605 0.43887844 0.85859792]
seed 42: [0.77395605 0.43887844 0.85859792]
seed 43: [0.65229926 0.04377532 0.02002959]
same seed, same numbers: True
```

The reason to avoid the global state is not style. Every function in the process uses the same global state. If one library function draws a number between your two draws, your sequence changes, and the cause is far from the symptom. A `Generator` that you create and pass to the functions that need it has no such coupling. The function signature shows that the function is random, and the caller controls the seed.

`np.random.default_rng`

```
np.random.default_rng(seed=None)
```

The function returns a `Generator`, the object that draws random numbers.

parameter	default	what it does	what goes wrong
`seed`	`None`	an integer, a list of integers, a `SeedSequence` or a generator. With `None`, fresh entropy from the operating system	with `None`, two runs give different numbers, and you cannot repeat the run

`np.random.Generator.random`

```
np.random.Generator.random(self, size=None, dtype=<class 'numpy.float64'>, out=None)
```

The method returns floats from the half-open range from 0.0 up to, and not including, 1.0.

parameter	default	what it does	what goes wrong
`size`	`None`	the shape of the result. With `None`, one number	a bare integer gives a 1-D array, and a tuple gives that shape
`dtype`	`<class 'numpy.float64'>`	`float64` or `float32`	—
`out`	`None`	an array to fill	—

`np.random.Generator.integers`

```
np.random.Generator.integers(self, low, high=None, size=None, dtype=<class 'numpy.int64'>,
endpoint=False)
```

The method returns random integers from the half-open range `[low, high)`.

parameter	default	what it does	what goes wrong
`low`	required	the smallest value. With `high` unset, it is the upper limit and the smallest value is 0	—
`high`	`None`	the upper limit. It is not included	`integers(0, 3)` never gives 3
`size`	`None`	the shape of the result	—
`endpoint`	`False`	with `True`, `high` is included	`integers(0, 3, endpoint=True)` can give 3. Without `endpoint`, the largest value is 2

Parallel streams: `spawn`, not `seed + i`

A job with several workers needs one stream per worker, and the streams must not overlap or repeat. The tool is `SeedSequence.spawn`.

```
import numpy as np


def work(seed_seq):
    return np.random.default_rng(seed_seq).normal(size=1000).mean()


children = np.random.SeedSequence(7).spawn(4)
forward = [work(s) for s in children]
backward = [work(s) for s in reversed(np.random.SeedSequence(7).spawn(4))][::-1]

print("four worker means:", [round(float(m), 4) for m in forward])
print("same in any order:", forward == backward)
print("different streams:", len(set(forward)) == 4)
```

```
four worker means: [-0.0682, 0.0625, 0.0279, -0.0366]
same in any order: True
different streams: True
```

`np.random.SeedSequence`

```
np.random.SeedSequence(entropy=None, *, spawn_key=(), pool_size=4, n_children_spawned=0)
```

The class turns a seed into the start states of one or more generators. It also makes child sequences.

parameter	default	what it does	what goes wrong
`entropy`	`None`	the seed: an integer or a list of integers. With `None`, fresh entropy	with `None`, the sequence cannot be repeated
`spawn_key`	`()`	the position of this sequence in a tree of sequences	a child made by `spawn` has a key such as `(0,)`, and two children must not share a key

`np.random.SeedSequence.spawn`

```
np.random.SeedSequence.spawn(self, n_children)
```

The method returns a list of child sequences. Each one has a different `spawn_key`, and so a different stream.

parameter	default	what it does	what goes wrong
`n_children`	required	how many children to make	calling it again on the same parent continues the count, so the next children are new ones, not the same ones

Each child sequence is derived from the parent in a way that is designed to give independent streams. The result of worker 2 does not depend on whether worker 1 ran first. This is the property that lets you move from a loop to a pool of processes and keep the numbers.

What the documentation promises, and what it does not

The numbers above are stable on one version of NumPy. The NumPy policy is that the streams of `Generator` methods are not guaranteed to be the same between versions. A seed makes a run reproducible on one installation. It does not make it reproducible for ever.

If a published number depends on a stream, write the NumPy version next to the seed. This book does so. NumPy 2.5.4 produced every output.

The function you build

A line fit gives one slope. To say how well the data fix the slope, resample the rows with replacement, fit again, and look at the spread of the slopes. This is the bootstrap. Doing 1000 fits is a loop in Python, and a few lines of broadcasting here.

```
import numpy as np


def bootstrap_slopes(x: np.ndarray, y: np.ndarray, n_boot: int, seed) -> np.ndarray:
    """Slope of the least-squares line for `n_boot` resamples of the rows, all at once."""
    g = np.random.default_rng(seed)
    idx = g.integers(0, len(x), (n_boot, len(x)))
    xs, ys = x[idx], y[idx]
    xc = xs - xs.mean(axis=1, keepdims=True)
    yc = ys - ys.mean(axis=1, keepdims=True)
    return (xc * yc).sum(axis=1) / (xc * xc).sum(axis=1)


def fit_line(x: np.ndarray, y: np.ndarray, n_boot: int = 1000, seed=0) -> tuple[float, float,
tuple[float, float]]:
    """Return (intercept, slope, (low, high)) with a 95% bootstrap interval for the slope."""
    X = np.column_stack([np.ones_like(x), x])
    (b0, b1), *_ = np.linalg.lstsq(X, y, rcond=None)
    low, high = np.percentile(bootstrap_slopes(x, y, n_boot, seed), [2.5, 97.5])
    return float(b0), float(b1), (float(low), float(high))


x = np.linspace(0, 1, 40)
y = 1 + 2 * x + np.random.default_rng(5).normal(0, 0.1, 40)

b0, b1, (lo, hi) = fit_line(x, y, seed=3)
print(f"intercept {b0:.3f}, slope {b1:.3f}, 95% interval for slope ({lo:.3f}, {hi:.3f})")
print("same seed, same interval:     ", fit_line(x, y, seed=3) == fit_line(x, y, seed=3))
print("different seed, same interval:", fit_line(x, y, seed=4)[2] == fit_line(x, y, seed=3)
[2])
print("true slope inside interval:   ", lo <= 2.0 <= hi)
```

```
intercept 1.001, slope 1.934, 95% interval for slope (1.850, 2.014)
same seed, same interval:      True
different seed, same interval: False
true slope inside interval:    True
```

The bootstrap function does three steps from earlier chapters. It draws a `(1000, 40)` array of row numbers. It uses an integer index list, from chapter 4, to build `x[idx]` of shape `(1000, 40)`. It uses `keepdims=True`, from chapter 5, to center each row. No loop appears.

`np.percentile`

```
np.percentile(a, q, axis=None, out=None, overwrite_input=False, method='linear',
keepdims=False, *, weights=None)
```

The function returns the value below which a given percent of the data lie.

parameter	default	what it does	what goes wrong
`a`	required	the data	—
`q`	required	the percent, from 0 to 100, as a number or a list	a fraction such as `0.025` means 0.025 percent. Use `2.5`
`axis`	`None`	the axis to work along. With `None`, the flattened array	—
`method`	`'linear'`	how to estimate between data points. There are many	`'nearest'` gives data values only, and `'linear'` can give a value that is not in the data

`hilbert`

```
hilbert(n: int) -> numpy.ndarray
```

The function returns the `n`-by-`n` Hilbert matrix, with the entry `1 / (i + j + 1)`.

parameter	default	what it does	what goes wrong
`n`	required	the size of the matrix	the condition number is 1.6e4 at 4, 1.5e10 at 8 and 1.8e16 at 12

`bootstrap_slopes`

```
bootstrap_slopes(x: numpy.ndarray, y: numpy.ndarray, n_boot: int, seed) -> numpy.ndarray
```

The function returns the slope of the least-squares line for each of `n_boot` resamples of the rows.

parameter	default	what it does	what goes wrong
`x`	required	the 1-D array of predictors	a constant `x` gives `nan` slopes and the warning `invalid value encountered in divide`
`y`	required	the 1-D array of observations, with the same length as `x`	a different length raises `IndexError`
`n_boot`	required	the number of resamples	the memory is `n_boot * len(x)` values for each of `x[idx]` and `y[idx]`
`seed`	required	any seed that `default_rng` accepts	the same seed gives the same slopes, and a different seed gives a different interval

`fit_line`

```
fit_line(x: numpy.ndarray, y: numpy.ndarray, n_boot: int = 1000, seed=0) -> tuple[float,
float, tuple[float, float]]
```

The function returns the intercept, the slope and a 95 percent bootstrap interval for the slope.

parameter	default	what it does	what goes wrong
`x`	required	the 1-D array of predictors	—
`y`	required	the 1-D array of observations	—
`n_boot`	`1000`	the number of resamples	with 500 resamples, the measured coverage was 289 of 300 trials
`seed`	`0`	the seed of the resampling	the default makes every call give the same interval, which hides the spread between seeds

Two checks that the module runs are worth stating. First, each bootstrap slope equals the slope that `lstsq` gives for the same resample, so the closed-form shortcut is right. Second, a bootstrap interval is a claim about coverage, and you can measure the claim. Fit 300 data sets that have a true slope of 2, build the interval for each, and count how often the interval holds 2.

Run this block from the root of the book, so that the import finds the module.

```
import numpy as np

from examples.ch07_linalg_random import fit_line

x = np.linspace(0, 1, 40)
trials, covered = 300, 0
for t in range(trials):
    y = 1 + 2 * x + np.random.default_rng(1000 + t).normal(0, 0.1, 40)
    _, _, (lo, hi) = fit_line(x, y, n_boot=500, seed=t)
    covered += lo <= 2.0 <= hi
print(f"covered {covered} of {trials}: {covered / trials:.3f}")
```

```
covered 289 of 300: 0.963
```

The interval is nominally 95 percent. For 300 trials, the standard error of a proportion near 0.95 is 0.013. A result between 0.92 and 0.98 agrees with the claim. The measured coverage is in that range.

How it works

Solve. The function `solve` does not compute an inverse. It factors the matrix into a lower and an upper triangle with row swaps, which is Gaussian elimination with partial pivoting. It then solves the two triangular systems. The next block writes this in a few lines and compares it with `np.linalg.solve`.

```
import numpy as np

from examples.ch07_linalg_random import hilbert


def lu_solve(A, b):
    """Gaussian elimination with partial pivoting, then back substitution."""
    A, b = A.astype(float).copy(), b.astype(float).copy()
    n = len(b)
    for k in range(n):
        p = k + np.argmax(np.abs(A[k:, k]))          # pivot: the largest value in the column
        if p != k:
            A[[k, p]], b[[k, p]] = A[[p, k]], b[[p, k]]
        for i in range(k + 1, n):
            f = A[i, k] / A[k, k]
            A[i, k:] -= f * A[k, k:]
            b[i] -= f * b[k]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - A[i, i + 1:] @ x[i + 1:]) / A[i, i]
    return x


rng = np.random.default_rng(0)
A, b = rng.random((50, 50)), rng.random(50)
mine, theirs = lu_solve(A, b), np.linalg.solve(A, b)
print("random 50 x 50, largest difference:", f"{np.abs(mine - theirs).max():.1e}")
print("random 50 x 50, my residual:       ", f"{np.abs(A @ mine - b).max():.1e}")

H = hilbert(8)
true = np.ones(8)
print("Hilbert 8, my error:     ", f"{np.abs(lu_solve(H, H @ true) - true).max():.1e}")
print("Hilbert 8, solve error:  ", f"{np.abs(np.linalg.solve(H, H @ true) - true).max():.1e}")
```

```
random 50 x 50, largest difference: 1.2e-14
random 50 x 50, my residual:        2.7e-15
Hilbert 8, my error:      7.0e-08
Hilbert 8, solve error:   1.2e-07
```

The hand-written version agrees with `solve` to rounding. On the Hilbert matrix both lose about the same number of digits: the errors are 7.0e-8 and 1.2e-7. The loss is probably the condition number at work and not a fault of the method. The two methods lose the same digits, which supports this, and this book did not isolate the cause. The inverse costs more because it solves for `n` right sides, one for each column of the identity, where `solve` has one.

Random numbers. A `Generator` holds a state, and each draw moves the state forward. The same seed gives the same state, so the same stream. A draw of three numbers and then a draw of three more equals one draw of six. The stream does not depend on how you cut it.

```
import numpy as np

g = np.random.default_rng(5)
state = g.bit_generator.state
print("generator:", state["bit_generator"])
print("state keys:", list(state))

one = np.random.default_rng(9)
two = np.random.default_rng(9)
cut = np.concatenate([one.random(3), one.random(3)])
whole = two.random(6)
print("3 + 3 draws equal 6 draws:", np.array_equal(cut, whole))

children = np.random.SeedSequence(7).spawn(3)
keys = [c.spawn_key for c in children]
starts = {str(np.random.default_rng(c).bit_generator.state["state"]["state"]) for c in
children}
print("spawn keys:", keys)
print("different start states:", len(starts), "of", len(children))

print("no seed gives different runs:", np.random.default_rng().random() !=
np.random.default_rng().random())
```

```
generator: PCG64
state keys: ['bit_generator', 'state', 'has_uint32', 'uinteger']
3 + 3 draws equal 6 draws: True
spawn keys: [(0,), (1,), (2,)]
different start states: 3 of 3
no seed gives different runs: True
```

Each child has its own key, and the key goes into the start state. This is why children do not repeat each other, and why the result for child 2 does not depend on child 1.

The invariant. The same seed and the same sequence of calls give the same numbers on one installation. A solve has a residual at the rounding level, whatever the condition number.

The failure. A solve with a large condition number has a small residual and a large error. A seed that you do not set gives a run that you cannot repeat.

Reference

Callable	Introduced in	Contract
`np.matmul`	The matrix product	matrix product, with broadcast over leading axes
`np.linalg.solve`	Solve, do not invert	solution of a square system by factorization
`np.linalg.cond`	Solve, do not invert	condition number of a matrix
`np.linalg.lstsq`	Fit with `lstsq`, not with the normal equations	least-squares solution, rank and singular values
`np.random.default_rng`	Random numbers: create a generator and pass it	a `Generator` from a seed
`np.random.Generator.random`	Random numbers: create a generator and pass it	floats from 0.0 up to, not including, 1.0
`np.random.Generator.integers`	Random numbers: create a generator and pass it	integers from `[low, high)`
`np.random.SeedSequence`	Parallel streams: `spawn`, not `seed + i`	seed to start states, and child sequences
`np.random.SeedSequence.spawn`	Parallel streams: `spawn`, not `seed + i`	independent child sequences
`np.percentile`	The function you build	value below which a percent of the data lie
`hilbert`	Solve, do not invert	the Hilbert test matrix
`bootstrap_slopes`	The function you build	slopes of many row resamples at once
`fit_line`	The function you build	line fit with a bootstrap interval

What you have now

You have `fit_line`, which fits a line and reports an interval, and which gives the same numbers for the same seed. You have three rules, each with a measurement behind it. `solve` has a residual at the rounding level when `inv` does not. `lstsq` keeps 7 digits that the normal equations lose at degree 9. A spawned seed sequence gives results that do not depend on the order of the workers.

Chapter 8 puts the whole book to work on one program. It ends with a list of the traps in the order that you meet them.

Chapter 8 — Putting it together

> **Build** — `kmeans`, a clustering routine of 20 lines with no Python loop over points, a loop version to check it, and a best-of-10 wrapper that fixes the one failure that the check finds.
>
> **Theory kept** — a vectorized program is a chain of the parts from earlier chapters, and each part has a cost in time, memory or precision. You test the chain against a slow, plain version of the same program.
>
> **Source** — New. The algorithm is Lloyd's k-means, which is standard. Every output below is from this book's code.
>
> **Run this first** — `uv run python -m examples.ch08_kmeans`, which is `examples/ch08_kmeans.py`.

The earlier chapters each gave one tool. This chapter uses four of them in one program and checks the result. The program is k-means clustering. It is small enough to read on one page. It needs a distance matrix (chapter 5), a group-by sum (chapter 4), a random generator that you pass (chapter 7) and the checks from chapters 2 and 3.

The algorithm in two lines

Start with `k` centroids. Repeat two steps until the labels stop changing. First, give each point the label of its nearest centroid. Second, move each centroid to the mean of the points that have its label.

The first step is a distance matrix and an `argmin`. The second step is a group-by sum divided by a count. Both are in the book already.

The function you build

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered      # chapter 5


def kmeans(X: np.ndarray, k: int, seed: int = 0, max_iter: int = 100) -> tuple[np.ndarray,
np.ndarray, int]:
    """Lloyd's algorithm. Returns (centroids, labels, iterations)."""
    g = np.random.default_rng(seed)                           # chapter 7
    C = X[g.choice(len(X), k, replace=False)].copy()          # chapter 3: copy, not a view
    labels = np.full(len(X), -1)
    for it in range(1, max_iter + 1):
        new_labels = dist_dot_centered(X, C).argmin(axis=1)   # chapter 5
        if np.array_equal(new_labels, labels):
            return C, labels, it - 1
        labels = new_labels
        sums = np.zeros_like(C)
        np.add.at(sums, labels, X)                            # chapter 4
        counts = np.bincount(labels, minlength=k)
        keep = counts > 0                                     # an empty cluster keeps its
centroid
        C[keep] = sums[keep] / counts[keep, None]             # chapter 5: broadcast a column
    return C, labels, max_iter


centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
g = np.random.default_rng(1)
X = np.concatenate([c + 0.8 * g.standard_normal((300, 2)) for c in centers])
truth = np.repeat(np.arange(3), 300)

C, labels, iters = kmeans(X, 3, seed=1)
print("iterations:", iters)
print("centroids:")
print(C.round(2))
print("cluster sizes:", np.bincount(labels))
```

```
iterations: 3
centroids:
[[ 5.97 -0.06]
 [-0.07  0.  ]
 [ 3.02  5.02]]
cluster sizes: [300 300 300]
```

The three true centers are `(0, 0)`, `(6, 0)` and `(3, 5)`. The centroids are close to these, in another order. Each cluster has about 300 points. The `for` loop runs over iterations, and there are few of them. No loop runs over points.

`kmeans`

```
kmeans(X: numpy.ndarray, k: int, seed: int = 0, max_iter: int = 100) -> tuple[numpy.ndarray,
numpy.ndarray, int]
```

The function runs Lloyd's algorithm. It returns the centroids, the label of each point and the number of iterations that changed a label.

parameter	default	what it does	what goes wrong
`X`	required	the points, as an `(n, d)` array	a 1-D array raises an error in the distance step. Use `X.reshape(-1, 1)` for 1-D points
`k`	required	the number of clusters	`k` above `n` raises `ValueError` at the choice of starting points
`seed`	`0`	the seed of the choice of starting points	seed 0 stops at purity 0.67 on the data of this chapter, and 24 of 30 seeds find the blobs
`max_iter`	`100`	the most iterations	at the limit the function returns with labels that may still change, and the third value equals `max_iter`

The function uses four more NumPy functions that need their signatures.

`np.random.Generator.choice`

```
np.random.Generator.choice(self, a, size=None, replace=True, p=None, axis=0, shuffle=True)
```

The method picks elements of an array, or integers below an integer, at random.

parameter	default	what it does	what goes wrong
`a`	required	an array, or an integer `n` for the values `0 .. n-1`	—
`size`	`None`	how many to pick	—
`replace`	`True`	with `False`, no element is picked twice	with `True`, two starting centroids can be the same point
`p`	`None`	the probability of each element. With `None`, all are equal	—

`np.argmin`

```
np.argmin(a, axis=None, out=None, *, keepdims=<no value>)
```

The function returns the index of the smallest value. If two values are equal, the first one wins.

parameter	default	what it does	what goes wrong
`a`	required	the array	—
`axis`	`None`	the axis to search along. With `None`, the flattened array	with `None` on an `(n, k)` matrix, you get one flat index and not one label for each point
`keepdims`	`<no value>`	with `True`, the axis stays with length 1	—

`np.zeros_like`

```
np.zeros_like(a, dtype=None, order='K', subok=True, shape=None, *, device=None)
```

The function returns an array of zeros with the shape of `a`.

parameter	default	what it does	what goes wrong
`a`	required	the array whose shape is copied	—
`dtype`	`None`	the type of the result. With `None`, the type of `a`	an integer `a` gives an integer result, which cannot hold a mean

`np.array_equal`

```
np.array_equal(a1, a2, equal_nan=False)
```

The function returns `True` if two arrays have the same shape and the same values.

parameter	default	what it does	what goes wrong
`a1`, `a2`	required	the two arrays	arrays of different shape give `False`, with no error
`equal_nan`	`False`	with `True`, two `nan` values count as equal	with `False`, an array that has a `nan` is never equal to itself

The comments name the chapter of each line. The line `C[keep] = sums[keep] / counts[keep, None]` has a trap that chapter 5 described. The count has shape `(k,)` and the sums have shape `(k, d)`. Without `None`, the shapes `(k, d)` and `(k,)` do not align, or they align wrongly when `d` equals `k`.

Check it against the slow version

A fast version that you did not check is a guess. The module has a second function, `kmeans_loop`, with the same algorithm and explicit Python loops, and the same starting centroids.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_loop, make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

C, labels, it = kmeans(X, 3, seed=1)
Cl, labels_l, it_l = kmeans_loop(X, 3, seed=1)
print("same labels:        ", np.array_equal(labels, labels_l))
print("same centroids:     ", np.allclose(C, Cl))
print("same iteration count:", it == it_l)
print("largest centroid difference:", np.abs(C - Cl).max())
```

```
same labels:         True
same centroids:      True
same iteration count: True
largest centroid difference: 0.0
```

`make_blobs`

```
make_blobs(n_per: int, centers: numpy.ndarray, spread: float, seed: int) ->
tuple[numpy.ndarray, numpy.ndarray]
```

The function makes `n_per` points around each center, with Gaussian noise. It returns the points and the true label of each point.

parameter	default	what it does	what goes wrong
`n_per`	required	the number of points for each center	—
`centers`	required	the centers, as a `(k, d)` array	—
`spread`	required	the standard deviation of the noise. The chapter uses 0.8	with the centers 5 or 6 apart, a spread above about 2 makes the blobs overlap. The book did not measure the limit
`seed`	required	the seed of the noise	the same seed gives the same points

`kmeans_loop`

```
kmeans_loop(X: numpy.ndarray, k: int, seed: int = 0, max_iter: int = 100) ->
tuple[numpy.ndarray, numpy.ndarray, int]
```

The function is `kmeans` with explicit Python loops. It is the check, and it is not for use on real data.

parameter	default	what it does	what goes wrong
`X`	required	the points, as an `(n, d)` array	slow: see the timing block, 900 points
`k`	required	the number of clusters	—
`seed`	`0`	the seed of the choice of starting points. The same seed gives the same start as `kmeans`	a different choice of start gives a different result, so compare at the same seed
`max_iter`	`100`	the most iterations	—

The two versions agree on every label, and the largest centroid difference is 0.0 on this input. On other data the two versions can add in a different order and differ at the level of rounding. This is why the check uses `np.allclose` for the centroids and not `==`. If one label differs, a tie that rounding broke the other way is a possible cause. This book did not test such a case. Compare the inertia then.

The check that found a real problem

Run the same function from 30 different seeds. The data are the same, and only the starting centroids differ. Count how often the run finds the three blobs, which we measure with *purity*: the fraction of points whose cluster has the same majority true label as they do.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_best, make_blobs, purity, inertia

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

single = [purity(kmeans(X, 3, seed=s)[1], truth, 3) for s in range(30)]
good = sum(p > 0.95 for p in single)
print(f"single start: {good} of 30 seeds find the blobs")
print("purity of the failures:", sorted({round(float(p), 2) for p in single if p <= 0.95}))

inertias = {}
for s in range(30):
    C, labels, _ = kmeans(X, 3, seed=s)
    inertias[s] = (purity(labels, truth, 3) > 0.95, inertia(X, C, labels))
ok = [v for good, v in inertias.values() if good]
bad = [v for good, v in inertias.values() if not good]
print(f"inertia of the {len(ok)} good runs:    {min(ok):.1f} to {max(ok):.1f}")
print(f"inertia of the {len(bad)} failed runs:  {min(bad):.1f} to {max(bad):.1f}")

C, labels, _ = kmeans(X, 3, seed=0)
print("seed 0: rows are clusters, columns are true blobs")
print(np.array([np.bincount(truth[labels == j], minlength=3) for j in range(3)]))
```

```
single start: 24 of 30 seeds find the blobs
purity of the failures: [0.67]
inertia of the 24 good runs:    1176.7 to 1176.7
inertia of the 6 failed runs:  6215.8 to 6503.7
seed 0: rows are clusters, columns are true blobs
[[  0 157   5]
 [  0 143   0]
 [300   0 295]]
```

In 6 of 30 runs, the algorithm stops at a purity of 0.67. The loop version agrees with the vectorized version on seed 1, so the cause is not a bug in the NumPy code. It is a property of the algorithm. The table for seed 0 shows the shape of the failure. One cluster holds all of blob 0 and nearly all of blob 2, so its centroid sits between them. The other two clusters split blob 1.

No single step improves this result. It is a local minimum.

Each run has an inertia, which is the sum of squared distances from the points to their centroids. The good runs all have the same inertia, and every failed run has a larger one. The inertia ranks the runs without the true labels, and this is why the standard fix works. Run the algorithm from several starts, and keep the run with the lowest inertia. The function `kmeans_best` does this with `SeedSequence.spawn`, as chapter 7 recommends.

```
import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_best, make_blobs, purity

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=1)

best = [purity(kmeans_best(X, 3, n_init=10, seed=s)[1], truth, 3) for s in range(30)]
print(f"best of 10 starts: {sum(p > 0.95 for p in best)} of 30 seeds find the blobs")
```

```
best of 10 starts: 30 of 30 seeds find the blobs
```

`kmeans_best`

```
kmeans_best(X: numpy.ndarray, k: int, n_init: int = 10, seed: int = 0) -> tuple[numpy.ndarray, numpy.ndarray, int]
```

The function runs `kmeans` from several starts and keeps the run with the lowest inertia.

parameter	default	what it does	what goes wrong
`X`	required	the points, as an `(n, d)` array	the cost is `n_init` times the cost of `kmeans`
`k`	required	the number of clusters	—
`n_init`	`10`	the number of starts	with 1, it is `kmeans`, and 24 of 30 seeds succeed. With 10, 30 of 30 succeed
`seed`	`0`	the seed of the starts. The starts come from `SeedSequence.spawn`	—

`inertia`

```
inertia(X: numpy.ndarray, C: numpy.ndarray, labels: numpy.ndarray) -> float
```

The function returns the sum of squared distances from each point to its centroid.

parameter	default	what it does	what goes wrong
`X`	required	the points	—
`C`	required	the centroids, as a `(k, d)` array	—
`labels`	required	the index of the centroid of each point	on the chapter data, the good runs have 1176.7 and the failed runs 6215.8 to 6503.7

`purity`

```
purity(labels: numpy.ndarray, truth: numpy.ndarray, k: int) -> float
```

The function returns the fraction of points whose cluster has the same majority true label as they do.

parameter	default	what it does	what goes wrong
`labels`	required	the cluster of each point	—
`truth`	required	the true label of each point, as non-negative integers	needs the true labels, so it cannot be used on real data
`k`	required	the number of clusters	—

With 10 starts, all 30 seeds find the blobs. If one start fails with probability 0.2, the chance that all 10 fail is 0.2 to the power 10, which is 1e-7. The 30 results agree with that arithmetic.

What the vectorization bought

```
import timeit

import numpy as np

from examples.ch08_kmeans import kmeans, kmeans_loop, make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
Xs, _ = make_blobs(300, centers, 0.8, seed=2)
t_loop = min(timeit.repeat(lambda: kmeans_loop(Xs, 3, seed=1), number=1, repeat=2))
t_vec = min(timeit.repeat(lambda: kmeans(Xs, 3, seed=1), number=1, repeat=5))
assert t_loop / t_vec > 20
print(f"900 points: loop {t_loop * 1e3:.0f} ms, vectorized {t_vec * 1e3:.1f} ms")

Xb, _ = make_blobs(100_000, centers, 0.8, seed=3)
t_big = min(timeit.repeat(lambda: kmeans(Xb, 3, seed=1), number=1, repeat=3))
print(f"300,000 points, vectorized: {t_big * 1e3:.0f} ms")
```

```
# output varies
900 points: loop 29 ms, vectorized 0.8 ms
300,000 points, vectorized: 320 ms
```

On 900 points the assert requires the vectorized version to be at least 20 times faster. The loop time scales with the number of points. At 300,000 points it is 333 times larger. The run above printed 29 ms for 900 points, so the estimate is about 10 seconds, and the run does not include it.

This is the usual pattern. The check runs on a small input, and the fast version runs on the real one.

How it works

Each iteration of `kmeans` is a chain of five steps, and every step has a shape. The next block runs one iteration by hand on the data of this chapter, and prints the shape of every value.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered
from examples.ch08_kmeans import make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, _ = make_blobs(300, centers, 0.8, seed=1)
k = 3
C = X[np.random.default_rng(1).choice(len(X), k, replace=False)].copy()

D = dist_dot_centered(X, C)
labels = D.argmin(axis=1)
sums = np.zeros_like(C)
np.add.at(sums, labels, X)
counts = np.bincount(labels, minlength=k)
new_C = sums / counts[:, None]

for name, value in [("X", X), ("C", C), ("D = distances", D), ("labels", labels),
                    ("sums", sums), ("counts", counts), ("counts[:, None]", counts[:, None]),
                    ("new centroids", new_C)]:
    print(f"{name:18s} {value.shape}")
print("values in D, the cost of the distance step:", D.size, "= n * k =", len(X) * k)
```

```
X                  (900, 2)
C                  (3, 2)
D = distances      (900, 3)
labels             (900,)
sums               (3, 2)
counts             (3,)
counts[:, None]    (3, 1)
new centroids      (3, 2)
values in D, the cost of the distance step: 2700 = n * k = 2700
```

The distance step has the cost `n * k * d` multiplications, and the group sum has the cost `n * d` additions. Both are linear in the number of points. The loop over iterations has few steps, so the total cost is linear in `n`.

The reason that the algorithm works is one fact. Each of the two steps does not make the inertia larger. The label step gives each point the nearest centroid. The mean step puts each centroid at the point that minimizes the squared distances of its cluster. The next block records the inertia after every label step for 30 seeds, and tests that it never rises.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered
from examples.ch08_kmeans import inertia, make_blobs

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, _ = make_blobs(300, centers, 0.8, seed=1)
k = 3

never_rises, iterations = 0, []
for seed in range(30):
    C = X[np.random.default_rng(seed).choice(len(X), k, replace=False)].copy()
    history = []
    for _ in range(100):
        labels = dist_dot_centered(X, C).argmin(axis=1)
        history.append(inertia(X, C, labels))
        sums = np.zeros_like(C)
        np.add.at(sums, labels, X)
        counts = np.bincount(labels, minlength=k)
        keep = counts > 0
        C[keep] = sums[keep] / counts[keep, None]
        if len(history) > 1 and np.isclose(history[-1], history[-2], rtol=0, atol=1e-9):
            break
    never_rises += all(b <= a + 1e-9 for a, b in zip(history, history[1:]))
    iterations.append(len(history))

print("seeds where the inertia never rose:", never_rises, "of 30")
print("iterations until it stopped: least", min(iterations), "most", max(iterations))
```

```
seeds where the inertia never rose: 30 of 30
iterations until it stopped: least 3 most 19
```

The invariant. The inertia never rises from one iteration to the next. The block above tests it on 30 starts.

The failure. The invariant says that the algorithm stops, and not that it stops at the best answer. It stops at the first point where no step helps, which is a local minimum. Six of the 30 seeds stop there. Their inertia is 5.3 to 5.5 times the best value, which is 1176.7.

The traps, in the order you meet them

Each row is a mistake that gives a wrong answer and no error. The chapter column says where this book measured it.

Trap	Why it happens	How to catch it	Chapter
An array has a different dtype than you assumed	NumPy picks the dtype from the data	Print `.dtype` at every input and output	2
Integers wrap	The dtype fixes the number of bits	`checked_add`, or a wider dtype	2
A `float32` running sum drifts	Small values are added to a large total	Use `x.sum()`, or `float64`	2
A function changes its argument	A slice is a view	`np.shares_memory`, `root`	3
A small view keeps a large block alive	The view holds a reference to its base	`.copy()`	3
A write is lost	A mask or index list makes a copy	Write with the mask on the left	4
A repeated index is applied once	`a[i] += 1` reads, adds, then writes	`np.add.at`, `np.bincount`	4
A result is `(n, n)` and not `(n,)`	Broadcasting stretches a length-1 axis	Assert the shape of a result	5
The distance matrix is wrong in `float32`	`|a|² + |b|² − 2a·b` subtracts large numbers	Center the data first	5
`np.vectorize` is no faster	It is a Python loop	Use ufuncs and `np.where`	6
`inv(A) @ b` is inaccurate	The inverse amplifies rounding	`solve`, and check the residual	7
A seed gives other numbers on another machine	The stream depends on the NumPy version	Write the version next to the seed	7
An iterative method stops at a poor answer	A local minimum	Several starts, keep the lowest inertia	8

The most useful single habit is the test in this chapter. Write the slow version first. It is short, it is obviously right, and it costs nothing to keep. Then write the fast version, and assert that the two agree on a small input.

What this book left out

This book is short, so it skips topics that have a real use. They are listed here so that you know where the edge is. This book did not measure any of them.

- `np.einsum` writes a sum over named axes in one call, and replaces many `@` and broadcast chains.
- Structured arrays and `np.recarray` hold records with named fields of mixed types.
- Masked arrays hold missing values with a mask and not a `nan`.
- `np.memmap` and `np.load(mmap_mode="r")` read an array from disk without loading it.
- `np.fft` computes Fourier transforms, and `np.linalg` has factorizations beyond those used here.
- Compilers such as Numba, and libraries such as SciPy, cover the loops that chapter 6 could not vectorize.

Reference

Callable	Introduced in	Contract
`kmeans`	The function you build	Lloyd's algorithm with no loop over points
`np.random.Generator.choice`	The function you build	random picks from an array, with or without replacement
`np.argmin`	The function you build	index of the smallest value along an axis
`np.zeros_like`	The function you build	zeros with the shape and type of another array
`np.array_equal`	The function you build	same shape and same values
`make_blobs`	Check it against the slow version	points around given centers, with the true labels
`kmeans_loop`	Check it against the slow version	the same algorithm with Python loops
`kmeans_best`	The check that found a real problem	the run with the lowest inertia of several starts
`inertia`	The check that found a real problem	sum of squared distances to the centroids
`purity`	The check that found a real problem	share of points in the majority cluster of their true label

What you have now

You have a clustering routine of 20 lines that you checked against a slow version, and a wrapper that fixes its one measured failure. You have a table of 13 traps, each with a measurement in an earlier chapter.

The book made one claim, and every chapter tested it: an array program is fast because of how it uses memory, and it is wrong in the places where it hides that use. The shape, the dtype and the strides from chapter 1 explain every row of the table above.

Appendix A — Described, not run

Every row below is a statement in the book that no block of the book tested. Each one is worded in the text as a statement of the documentation, as a likely cause, or as a limit of the book. Nothing in this list is a measured result.

Chapter	Statement	Where it comes from	How you can test it
1	NumPy 1.x made an array of objects from a list of unequal rows	memory of the NumPy 1.x behavior	install NumPy 1.26 and run `np.array([[1, 2], [3]])`
1	a list holds pointers to separate integer objects, and a sum must follow each pointer	the CPython object model	read `sys.getsizeof` of the list and of one item, and time `sum` on a list of floats
2	NumPy 1.x chose the result dtype from the value of a Python number	the NumPy documentation of NEP 50	install NumPy 1.26 and run `np.uint8(1) + 300`
2	`x.sum()` adds in pairs, and that is why it is accurate	the NumPy documentation of `np.sum`	add the values in a Python loop and in a pairwise tree, and compare both with the exact sum
3	a strided read is slow because it wastes the line of 64 bytes that a read brings in	the usual model of a CPU cache	read with strides of 1, 2, 4, 8, 16 and 32 values, and look for steps in the time
3	the result of `np.broadcast_to` is read-only because elements share a location	the NumPy documentation of `np.broadcast_to`	none: the read-only flag is printed in chapter 3
3	the `OWNDATA` flag tells which array frees the block	the NumPy documentation of `ndarray.flags`	none that is simple
4	`np.add.at` was slow in NumPy 1.x	memory of release notes	install NumPy 1.26 and time the block of chapter 4
5	the dot version is fast because it never forms the `(n, m, d)` array	a likely cause	time a version that forms the array and one that does not, with the same arithmetic
6	a ufunc picks the first loop to which both inputs can be cast safely, and casts in small buffers	the NumPy documentation of ufuncs	`np.add.resolve_dtypes`, and the `bufsize` of `np.nditer`
6	the window version is slower because it takes a real mean over each row	a likely cause	count the additions of the window version and of the cumulative-sum version
6	an exponential moving average can be written with cumulative sums and powers	the algebra of the recurrence	write it, and compare it with the loop on 1e6 values
6	a filter routine and Numba are faster than the loop	memory	time both on the loop of chapter 6
7	the loss of digits of the normal equations is the squared condition number	the theory of least squares	the column `cond(XtX)` in the table of chapter 7 is consistent with it. Vary the degree finely and fit the slope
7	the Hilbert matrix loses the same digits in `solve` and in `lu_solve` because of its condition number	a likely cause	compare both on a well-conditioned matrix of the same size
7	the streams of `Generator` methods can change between NumPy versions	the NumPy policy for random numbers	install two versions and compare the first values for one seed
7	`spawn_key` goes into the start state	the NumPy documentation of `SeedSequence`	the block of chapter 7 shows that the start states differ, not why
8	a label that differs between `kmeans` and `kmeans_loop` is a tie that rounding broke	a possible cause	build a data set with exact ties and compare

Chapter	Statement	Where it comes from	How you can test it
8	blobs overlap when the spread is above about 2	a rough guess, not measured	run `kmeans_best` for spreads from 0.5 to 4 and plot the purity
8	k-means++ needs fewer restarts	memory	implement it and count the failures on the data of chapter 8

Appendix B — What went wrong while writing

This list is the record of statements that the author wrote, and that a run or an audit then showed to be wrong. They are fixed in the text. They are listed here because the same habits come back.

Chapter	The wrong statement	What the run or the audit showed	How it was caught
1	an array made with `reshape` owns its block	`owns=False`: it is a view of the array that `arange` made	the first output of `describe`
2	the `float32` cumulative sum is 8.8 percent too small	it is 8.8 percent too large	reading the printed total, 1087937 against 1000000
2	each add of 0.1 is "below the gap and partly lost"	the add is rounded in both directions: -6 percent at one total, +25 percent at another	a block that measures one add at five totals
3	a view of a view points to the array one step up	NumPy points `.base` straight to the owner	printing `.base` of a view of a view
3	the quick test never says no when the answer is yes	untested at first. It holds on 5000 random slice pairs: 0 cases of the other kind	a block that counts both kinds of error
5	the mean distance of the points is 6.6e-3 and the centered error is 8e-11	6.9e-3 and 1.2e-10	the printed output
5	the gap of neighbor `float32` values "is as large as the whole cube"	the gap is one tenth of the side of the cube	arithmetic, then the printed norms
6	"about 90 ufuncs"	106 in the `numpy` namespace	counting them in a block
6	the centered cumulative sum has an error of about 1e-4 on seed 3	4.7e-4. On seed 2 it is exactly 0, and the reason is the size of the total	the printed output and the largest centered total
6	"there is no accumulate form" of the exponential moving average	a form with cumulative sums and powers exists	reading the claim as an enemy
7	the normal equations lose 8 digits at degree 9	7 digits: 1.3e-3 against 5.4e-11	the printed output
7	`inv` has a residual 1e8 times larger at `n = 8`	6e7 times	the printed output
7	`solve` takes 0.18 s and `inv` 0.48 s for 2000	the ratio changes with the load of the machine and with the number of threads, down to 0.88	the number audit, and a run on a busy machine
8	the first k-means run finds the three blobs	purity 0.67: a local minimum, on seed 0	the assert of the first draft failed
8	the guessed timings 160 ms and 4.6 ms	29 ms and 0.8 ms	running the block
all	timings typed into `# output varies` blocks	they came from other scripts. They are now printed by the block itself	`scripts/refresh_varies.py`
all	a timing assert passed when blocks ran in parallel and failed under load	the checker now runs timed blocks alone	a failed run, then the checker rule

Checks that found nothing

Four checks found no error in this book at the last run.

- The link check.
- The signature check of every `####` heading.
- The check that every `np.*` name in the text exists.
- The check that the defaults in the parameter tables are the defaults of the installed NumPy.

Exercises — Chapter 1

Chapter: The array · module: `examples/ch01_array.py` · solutions: 01

1. [measure] Make `np.zeros((100, 200, 3), dtype=np.float32)`. Report its `strides` and its `nbytes`. Expected: strides `(2400, 12, 4)` and `nbytes` 240000, exact.

2. [break] Chapter 1 says that a transpose is not C-contiguous. Find a 2-D array for which the transpose is still C-contiguous. Explain why the flag stays `True`. Expected: a shape with an axis of length 1, for example `(1, 5)`.

3. [judge] A colleague stores 20 sensor readings per request in a NumPy array, and the service handles thousands of requests each second. Another colleague says that a plain list is better. What does a good answer weigh? (Consider the fixed cost per call from chapter 1, what the readings are used for next, and what a measurement on the real service must show.)

Exercises — Chapter 2

Chapter: Dtypes, and the answers that change silently · module: `examples/ch02_dtypes.py` · solutions: 02

1. [measure] Find the smallest positive integer `n` for which `np.float32(n) + np.float32(1) == np.float32(n)`. Expected: 16777216, exact.

2. [break] The cumulative sum of ten million `float32` values of 0.1 is wrong by 8.8 percent in chapter 2. Change one argument of `np.cumsum`, and nothing else, so that the last value is within 1e-6 (relative) of 1,000,000.015. Expected: `dtype=np.float64`, and a relative error below 1e-6.

3. [judge] A library function adds two `uint8` images. What must it do on overflow: wrap, clip at 255, or raise an error? What does a good answer weigh? (Consider who calls the function, what a wrapped pixel looks like, and what each choice costs in speed and in surprise.)

Exercises — Chapter 3

Chapter: Views and copies · module: `examples/ch03_views.py` · solutions: 03

1. [measure] Make `big = np.zeros((1000, 1000))` and `small = big[:10]`. How many bytes does `small` keep alive through `small.base`? Expected: 8000000, exact.

2. [break] The function `center` in chapter 3 changes the column that the caller passes in. Change one line so that the caller's data stay the same, and show that they do. Expected: the original array equals its copy from before the call.

3. [judge] You write a public function that takes an array and returns a part of it. Does it return a view or a copy? What does a good answer weigh? (Consider memory, the caller's later writes, and what the documentation can promise.)

Exercises — Chapter 4

Chapter: Indexing · module: `examples/ch04_indexing.py` · solutions: 04

1. [measure] Let `idx = np.array([0, 0, 0, 1, 1, 2])`. Add 1 at each index of `np.zeros(3, dtype=int)` in two ways.

- Use `a[idx] += 1`.
- Use `np.add.at`.

Report both results. Expected: `[1 1 1]` and `[3 2 1]`, exact.

2. [break] The `np.where(x > 0, np.log(x), 0)` call in chapter 4 gives a warning and not an error. Wrap the call so that it raises `FloatingPointError`, and say what that shows about `np.where`. Expected: the message `divide by zero encountered in log`.

3. [judge] A team uses `np.add.at`. They say that `np.bincount` is "known to be faster". Chapter 4 measured the two as within a factor of 2 on one version of NumPy. What does a good answer weigh before the team changes the code?

Consider the version, the key range, the profile of their own data, and the cost of the change.

Exercises — Chapter 5

Chapter: Broadcasting · module: `examples/ch05_broadcasting.py` · solutions: 05

1. [measure] Use `tracemalloc` to measure the peak memory of `dist_broadcast` for two sets of 1000 points in 32 dimensions. Expected: 264 MB, within ±10 percent. State the number of values in the temporary array and the result of the arithmetic.

2. [break] Make `dist_dot` return a maximum error larger than 1.0 for `float64` points that lie in a cube of side 1. Change only the position of the cube. Expected: an offset of 1e8 gives an error of about 4. Explain the result with the gap between neighbor `float64` values.

3. [judge] Your service computes distances for 50 points against 50,000 points in 128 dimensions, in `float32`, on a machine with 8 GB. Which of the three versions do you ship? What does a good answer weigh? (Consider the size of the temporary array, the precision, and whether centering is possible when the second set changes at each call.)

Exercises — Chapter 6

Chapter: Ufuncs and vectorization · module: `examples/ch06_ufuncs.py` · solutions: 06

1. [measure] For 100,000 values, measure the ratio of the time of `ma_window` to the time of `ma_cumsum` at window 10 and at window 1000. Expected: the ratio at window 1000 is above 10, and it is larger than the ratio at window 10. Say why the ratio depends on the window.

2. [break] `np.vectorize(lambda v: 0 if v < 1 else v)(np.array([0.5, 2.7]))` must give `[0. 2.7]`. Run it and report the result. Then repair it with one argument. Expected: the result `[0 2]` first, and `[0. 2.7]` after the repair.

3. [judge] A colleague wants to rewrite the exponential moving average of chapter 6 in Numba. What does a good answer weigh? (Consider the 3.1 million steps a second that one run of chapter 6 printed, the size of the real data, the cost of a new dependency, and what a profile must show first.)

Exercises — Chapter 7

Chapter: Linear algebra and random numbers · module: `examples/ch07_linalg_random.py` · solutions: 07

1. [measure] Compute `np.linalg.cond` of the 10-by-10 Hilbert matrix. Expected: 1.6e13, within a factor of 2.

2. [break] `np.linalg.solve` gave an error of 1.2e-7 for the 8-by-8 Hilbert system in `float64`. Change only the dtype of the inputs so that the error is larger than 1. Expected: `float32` gives an error near 7. Explain the result with the condition number and the `float32` epsilon.

3. [judge] A team of five runs experiments. Each script has `np.random.seed(0)` at the top. The scripts import shared helper functions that also draw random numbers. What does a good answer weigh before the team changes its practice? (Consider hidden coupling, reproducibility across versions, and the cost of passing a generator through every function.)

Exercises — Chapter 8

Chapter: Putting it together · module: `examples/ch08_kmeans.py` · solutions: 08

1. [measure] Use `make_blobs(300, centers, 0.8, seed=2)` with the centers of chapter 8. Run `kmeans` once from each of the seeds 0 to 29. Count the runs that have a purity of 0.95 or less. Expected: 5, exact.

2. [break] Replace the starting centroids in `kmeans` with the view `X[:k]`, and remove the `.copy()`. Run the function and report whether the input `X` changed. Expected: `X` changed. Explain which chapter predicted it.

3. [judge] Best-of-10 starts costs ten times as much as one start. A colleague says that a smarter initialization such as k-means++ makes it unnecessary. What does a good answer weigh? (Consider the measured failure rate of 6 in 30, the cost per run at your data size, and what you can measure on your data before you decide.)

Solutions — Chapter 1

Exercises: 01 · chapter: The array

1. [measure]

The strides are the number of bytes to jump along each axis. The last axis has 3 values of 4 bytes, so its stride is 4. The middle axis jumps over one row of 3 values, which is 12 bytes. The first axis jumps over 200 rows of 12 bytes, which is 2400 bytes.

```
import numpy as np

a = np.zeros((100, 200, 3), dtype=np.float32)
print(a.strides)
print(a.nbytes)
assert a.strides == (2400, 12, 4)
assert a.nbytes == 100 * 200 * 3 * 4
```

```
(2400, 12, 4)
240000
```

2. [break]

The transpose of a `(1, 5)` array is a `(5, 1)` array. An axis of length 1 is never stepped along, so its stride does not matter, and NumPy sets the contiguous flag from the other axes.

```
import numpy as np

a = np.zeros((1, 5))
print("a   shape", a.shape, "strides", a.strides, "contiguous", a.flags.c_contiguous)
print("a.T shape", a.T.shape, "strides", a.T.strides, "contiguous", a.T.flags.c_contiguous)
assert a.T.flags.c_contiguous
assert np.shares_memory(a, a.T)
```

```
a   shape (1, 5) strides (40, 8) contiguous True
a.T shape (5, 1) strides (8, 40) contiguous True
```

The strides changed from `(40, 8)` to `(8, 40)`, and the array is still contiguous. The flag describes the memory order that a walk follows. A walk over a `(5, 1)` array has only one axis that moves.

3. [judge]

Position. Keep the list, unless a measurement on the real service shows that the array helps.

What a good answer weighs. Twenty values is near the size where the fixed cost of an array call is larger than the work. The chapter measured a loss for three values. At thousands of requests each second, the cost per request matters.

The next step of the readings is the second fact. If the next step is a mean and a threshold, a list is enough. If the next step is a matrix product with a model, the model needs an array, and the conversion cost is paid once. The measurement must use the real request rate and the real next step. A benchmark of 20 values alone is not enough.

Strongest objection. The array gives the same code path as the offline batch job that runs on millions of readings. One code path has fewer bugs. This is an argument about maintenance and not about speed.

What overturns the position. A profile can show that the conversion costs less than 5 percent of the request time. A requirement to share code with the batch job has the same effect.

Solutions — Chapter 2

Exercises: 02 · chapter: Dtypes, and the answers that change silently

1. [measure]

A `float32` has 24 bits of precision. Below 2^{24} every integer is exact and has a gap of at most 1 to its neighbor. At 2^{24} the gap becomes 2, and adding 1 rounds back to the same number.

```
import numpy as np

n = np.arange(1, 2 ** 24 + 2, dtype=np.float32)        # every integer up to 2**24 + 1
stuck = np.flatnonzero(n + np.float32(1) == n)
smallest = int(n[stuck[0]])
print(smallest)
assert smallest == 2 ** 24 == 16777216
```

```
16777216
```

The test runs on every integer from 1 to $2^{24} + 1$, so it finds the smallest `n` and not only a power of 2.

2. [break]

The argument is `dtype=np.float64`. The cumulative sum then keeps its running total in `float64`, which has digits to spare.

```
import numpy as np

x = np.full(10_000_000, 0.1, dtype=np.float32)
exact = float(np.float32(0.1)) * x.size

plain = float(np.cumsum(x)[-1])
wide = float(np.cumsum(x, dtype=np.float64)[-1])
print(f"plain cumsum relative error: {abs(plain - exact) / exact:.2e}")
print(f"float64 cumsum relative error: {abs(wide - exact) / exact:.2e}")
assert abs(plain - exact) / exact > 1e-2
assert abs(wide - exact) / exact < 1e-6
```

```
plain cumsum relative error: 8.79e-02
float64 cumsum relative error: 0.00e+00
```

The input is still `float32`. Only the accumulator changed. The cost is memory: the result of `cumsum` is now 80 MB and not 40 MB.

3. [judge]

Position. Raise an error by default. Offer a `clip=True` option for the caller that wants saturation. Wrapping is almost never what the caller means.

What a good answer weighs. The caller of an image function usually wants a bright pixel to stay bright. That is a clip. A wrapped pixel turns a bright area dark, and the image looks plausible and is wrong. An error stops the program at the first bad input. The cost is one extra comparison pass.

A clip hides that the input was out of range, and that fact can be the information that the caller needs. Wrapping is correct for hash functions and modular counters. These are not images.

Strongest objection. In a video pipeline that runs at 60 frames a second, an error stops the stream on one bad frame. For this caller the silent clip is the safe choice. The answer is the option: the default is the error, and the pipeline sets `clip=True`.

What overturns the position. A measurement can show that the check costs more than the budget of the caller. An example is 10 percent of the frame time. Then the default must change, and the first line of the documentation must say so.

Solutions — Chapter 3

Exercises: 03 · chapter: Views and copies

1. [measure]

```
import numpy as np

big = np.zeros((1000, 1000))
small = big[:10]
print("small owns:", small.nbytes, "bytes")
print("small keeps:", small.base.nbytes, "bytes alive")
assert small.base.nbytes == 8_000_000
assert small.nbytes == 80_000
```

```
small owns: 80000 bytes
small keeps: 8000000 bytes alive
```

The slice has 10 rows of 1000 values of 8 bytes, which is 80,000 bytes. The base has 1000 rows, which is 8,000,000 bytes. The ratio is 100.

2. [break]

Replace `x -= x.mean()` with `x = x - x.mean()`. The second form makes a new array and binds the name `x` to it. The array of the caller is not touched.

```
import numpy as np


def center(x):
    x = x - x.mean()
    return x


data = np.array([[1.0, 10.0], [3.0, 20.0], [5.0, 30.0]])
before = data.copy()
centered = center(data[:, 0])
print("data unchanged:", np.array_equal(data, before))
print("result:", centered)
assert np.array_equal(data, before)
```

```
data unchanged: True
result: [-2.  0.  2.]
```

3. [judge]

Position. Return a view when the function is a selection that the caller keeps in step with the original. Return a copy when the function computes something. Write the choice in the first line of the docstring.

What a good answer weighs. A view costs nothing and keeps the base alive, which can be a large block. A copy costs memory and time and breaks the link. A later write by the caller does not change the original.

The danger of a view is a write that the caller does not expect. The danger of a copy is a write that the caller expects and does not get. NumPy returns views for slices and copies for index lists, so a caller who knows NumPy expects the same pattern.

Strongest objection. A function that returns a view or a copy depending on the input, like `reshape`, is a trap. It is better to always copy. This is correct for a public function. The cost is memory on large arrays, and the answer is to document the exception where the cost is measured.

What overturns the position. A user report of a lost write or an unexpected change shows that callers do not read the docstring. Then return copies, and add a separate function that returns a view.

Solutions — Chapter 4

Exercises: 04 · chapter: Indexing

1. [measure]

```
import numpy as np

idx = np.array([0, 0, 0, 1, 1, 2])

a = np.zeros(3, dtype=int)
a[idx] += 1
b = np.zeros(3, dtype=int)
np.add.at(b, idx, 1)
print("a[idx] += 1:", a)
print("np.add.at  :", b)
assert a.tolist() == [1, 1, 1]
assert b.tolist() == [3, 2, 1]
```

```
a[idx] += 1: [1 1 1]
np.add.at  : [3 2 1]
```

Index 0 appears 3 times and index 1 appears 2 times. The statement `a[idx] += 1` reads `a[idx]` once, so each position gets only one update.

2. [break]

`np.errstate(all="raise")` turns every floating-point warning into an exception in its block.

```
import numpy as np

x = np.array([1.0, 0.0])
with np.errstate(all="raise"):
    try:
        np.where(x > 0, np.log(x), 0)
    except FloatingPointError as e:
        print(type(e).__name__, "-", e)
```

```
FloatingPointError - divide by zero encountered in log
```

The error comes from `np.log(x)`, which `np.where` receives as an argument. The condition `x > 0` is false for the second element, and it does not stop the log from running. This shows that `np.where` selects among values that are already computed.

3. [judge]

Position. Do not change the code until a profile of the real data shows that this call is a large part of the run time.

What a good answer weighs. The advice is old. It dates from NumPy 1.x, which the book did not test. On 2.5.4 the two are within a factor of 2 for the input of chapter 4. The assert of chapter 4 proves it. The version of the team is the first fact to check.

The key range is the second fact. The function `np.bincount` needs non-negative integers and makes an array as long as the largest key. That is wasteful for sparse keys. The third fact is how much of the run time the call takes. A change that saves 20 percent of 2 percent of the run is not worth a review.

Strongest objection. The function `np.bincount` states the intent more directly, and it reads better. This is a fair reason to change. It is a reason about readability, and it needs no speed claim.

What overturns the position. A profile that shows the call above 20 percent of the run time overturns it. A measurement on the team's own data and version that shows `bincount` is faster by a useful margin has the same effect.

Solutions — Chapter 5

Exercises: 05 · chapter: Broadcasting

1. [measure]

The temporary array has `n * m * d` values: 1000 × 1000 × 32 = 32,000,000. At 8 bytes each that is 256 MB. The result adds 1000 × 1000 × 8 bytes, which is 8 MB. The sum is 264 MB.

```
import tracemalloc

import numpy as np

from examples.ch05_broadcasting import dist_broadcast

A = np.random.default_rng(0).random((1000, 32))
tracemalloc.start()
dist_broadcast(A, A)
_, peak = tracemalloc.get_traced_memory()
tracemalloc.stop()
print("values in the temporary array:", 1000 * 1000 * 32)
print("expected MB:", (1000 * 1000 * 32 + 1000 * 1000) * 8 / 1e6)
print("measured peak MB:", round(peak / 1e6))
assert abs(peak / 1e6 - 264) < 26
```

```
values in the temporary array: 32000000
expected MB: 264.0
measured peak MB: 264
```

2. [break]

At an offset of 1e8, the squared norms are about 3e16. The gap between neighbor `float64` values at 3e16 is 4. Each term of the subtraction `|a|² + |b|² − 2a·b` has a rounding error of about one gap. The squared distance is wrong by a few gaps, about 16. The square root of 16 is 4.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot, dist_loop

for offset in (1e4, 1e6, 1e8):
    P = np.random.default_rng(0).random((30, 3)) + offset
    err = np.abs(dist_dot(P, P) - dist_loop(P, P)).max()
    print(f"offset {offset:8.0e}  max error {err:.2e}")
assert err > 1.0
print("gap at 3e16:", np.spacing(3e16))
```

```
offset    1e+04  max error 3.45e-04
offset    1e+06  max error 3.12e-02
offset    1e+08  max error 4.00e+00
gap at 3e16: 4.0
```

The points lie in a cube of side 1, so the true distances are at most 1.7. An error of 4 is larger than every true distance. Centering removes the problem, because it moves the cube to the origin before the subtraction.

3. [judge]

Position. Ship the dot version with centering. Process the 50,000 points in blocks. Test it against the broadcast version on a sample.

What a good answer weighs. The broadcast version needs 50 × 50,000 × 128 × 4 bytes, which is 1.28 GB in `float32`. The machine has 8 GB. One request fits, and a few at the same time fail. The dot version needs about the size of the result, which is 10 MB.

Precision is the open question. Centering with the mean of the 50,000 fixed points works if they do not change between calls. If both sets change, the mean of the 50 query points is a cheaper way to center. A fixed reference vector also works.

Strongest objection. The dot version in `float32` loses digits even after centering, if the points are spread over a large range. The answer is to measure the error that the service cares about. For example, measure whether the nearest neighbor is the same one. Do not measure the distance error.

What overturns the position. Compare with the broadcast version in `float64` on real data. Suppose the nearest neighbor changes for more than a small fraction of the queries. Then use `float64` for the cross term, or compute the final candidates exactly.

Solutions — Chapter 6

Exercises: 06 · chapter: Ufuncs and vectorization

1. [measure]

```
import timeit

import numpy as np

from examples.ch06_ufuncs import ma_cumsum, ma_window

x = np.random.default_rng(0).random(100_000)
ratio = {}
for w in (10, 1000):
    t_window = min(timeit.repeat(lambda: ma_window(x, w), number=1, repeat=3))
    t_cumsum = min(timeit.repeat(lambda: ma_cumsum(x, w), number=1, repeat=3))
    ratio[w] = t_window / t_cumsum
assert ratio[1000] > 10
assert ratio[1000] > ratio[10]
print("window time grows with w:", ratio[1000] > ratio[10])
print(f"ratio at w=10: {ratio[10]:.0f}, at w=1000: {ratio[1000]:.0f}")
```

```
# output varies
window time grows with w: True
ratio at w=10: 2, at w=1000: 27
```

The window version does `w` additions for each output value. The cumulative-sum version does two, whatever the size of `w`. The ratio therefore grows with `w`. The asserts prove only that the ratio at `w = 1000` is above 10 and above the ratio at `w = 10`.

2. [break]

`np.vectorize` finds the output type by calling the function on the first element. The first call returns the integer 0, so the output type is `int`, and the value 2.7 is cut to 2. NumPy gives no warning.

```
import numpy as np

f = lambda v: 0 if v < 1 else v
broken = np.vectorize(f)(np.array([0.5, 2.7]))
fixed = np.vectorize(f, otypes=[float])(np.array([0.5, 2.7]))
print("broken:", broken)
print("fixed: ", fixed)
assert broken.tolist() == [0, 2]
assert fixed.tolist() == [0.0, 2.7]
```

```
broken: [0 2]
fixed:  [0.  2.7]
```

The repair is `otypes=[float]`. This is a second reason, after speed, to prefer `np.where` over `np.vectorize`: `np.where(v < 1, 0.0, v)` has no output type to guess.

3. [judge]

Position. Not yet. First measure the run time on the real data.

What a good answer weighs. The loop ran at 3.1 million steps a second in the run of chapter 6. A series of one million values takes 0.3 seconds. A series of 100,000 takes 0.03 seconds. If the job runs once an hour, this is not a problem. Numba adds a dependency, a compile step at startup and a second place for bugs.

If a request has 10,000 series of 100,000 values each, the same arithmetic gives 5 minutes. Then the speed matters. The size of the real data and the time budget decide. The chapter measured only the speed of the loop.

Strongest objection. A library routine for linear filters applies this recurrence in compiled code, with no new compiler. That is a better first step than Numba. This book did not measure it. A good answer says so and measures it before the choice.

What overturns the position. A profile of the real job can show the loop above the time budget. A measurement can then show that the library routine does not meet the budget either. Then Numba is the next step.

Solutions — Chapter 7

Exercises: 07 · chapter: Linear algebra and random numbers

1. [measure]

```
import numpy as np

from examples.ch07_linalg_random import hilbert

c = np.linalg.cond(hilbert(10))
print(f"cond(H10) = {c:.2e}")
assert 0.8e13 < c < 3.2e13
```

```
cond(H10) = 1.60e+13
```

The textbook value is 1.6e13. With a rounding unit of 1.1e-16, the rule of chapter 7 predicts an error of about 2e-3 for `solve` on this system.

2. [break]

The `float32` epsilon is 1.2e-7. The condition number of the 8-by-8 Hilbert matrix is 1.5e10. The product is 1.8e3, which is more than the size of the answer. No digit is guaranteed, and the measured error is of order 1.

```
import numpy as np

from examples.ch07_linalg_random import hilbert

H = hilbert(8)
b = H @ np.ones(8)
e64 = np.abs(np.linalg.solve(H, b) - 1).max()
e32 = np.abs(np.linalg.solve(H.astype(np.float32), b.astype(np.float32)) - 1).max()
print(f"float64 error: {e64:.1e}")
print(f"float32 error: {e32:.1f}")
assert e64 < 1e-5
assert e32 > 1.0
print("cond * eps32 =", f"{np.linalg.cond(H) * np.finfo(np.float32).eps:.1e}")
```

```
float64 error: 1.2e-07
float32 error: 6.8
cond * eps32 = 1.8e+03
```

The rule of thumb gives an upper bound and not a prediction. The measured error is 6.8, which is below the bound of 1.8e3 and far above 1.

3. [judge]

Position. Stop using the global seed. Create one generator in the main script. Pass it to every function that draws numbers.

What a good answer weighs. The shared helpers draw from the same global state. A change in one helper changes the numbers in every script that calls it, and nobody sees the link. A passed generator makes the dependency visible in the signature.

The cost is one argument on each function and one decision about who owns the seed. For parallel work, add `SeedSequence.spawn`. No method of seeding promises the same stream across NumPy versions, so each result must also record the version.

Strongest objection. Five scripts already work. A change of every signature touches all of them for no visible gain. This is true for the code that exists. The practical answer is to apply the rule to new code and to the helpers that you change anyway.

What overturns the position. A search of the helpers for `np.random` calls takes minutes. If no helper draws random numbers, the global seed has no hidden coupling, and the change gives little.

Solutions — Chapter 8

Exercises: 08 · chapter: Putting it together

1. [measure]

```
import numpy as np

from examples.ch08_kmeans import kmeans, make_blobs, purity

centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, truth = make_blobs(300, centers, 0.8, seed=2)
failures = [s for s in range(30) if purity(kmeans(X, 3, seed=s)[1], truth, 3) <= 0.95]
print("failed seeds:", failures)
print("count:", len(failures))
assert len(failures) == 5
```

```
failed seeds: [0, 6, 13, 15, 19]
count: 5
```

The data of seed 2 give 5 failures in 30. The data of seed 1 in chapter 8 give 6. The failure rate depends on the data, and this is why the chapter reports a count and not a probability.

2. [break]

A basic slice is a view. The first `kmeans` step then writes the new centroids into `C`, which is the first `k` rows of `X`. The input changes, and the three rows that changed are the first three rows of `X`.

```
import numpy as np

from examples.ch05_broadcasting import dist_dot_centered
from examples.ch08_kmeans import make_blobs


def kmeans_view(X, k, max_iter=100):
    C = X[:k]                                   # a view: the .copy() is gone
    labels = np.full(len(X), -1)
    for _ in range(max_iter):
        new_labels = dist_dot_centered(X, C).argmin(axis=1)
        if np.array_equal(new_labels, labels):
            break
        labels = new_labels
        sums = np.zeros_like(C)
        np.add.at(sums, labels, X)
        counts = np.bincount(labels, minlength=k)
        keep = counts > 0
        C[keep] = sums[keep] / counts[keep, None]
    return C, labels


centers = np.array([[0.0, 0.0], [6.0, 0.0], [3.0, 5.0]])
X, _ = make_blobs(300, centers, 0.8, seed=1)
before = X.copy()
C, labels = kmeans_view(X, 3)
print("C shares memory with X:", np.shares_memory(C, X))
print("X unchanged:", np.array_equal(X, before))
print("rows that changed:", int((X != before).any(axis=1).sum()))
assert not np.array_equal(X, before)
```

```
C shares memory with X: True
X unchanged: False
rows that changed: 3
```

Chapter 3 predicted this: the rule is that basic slicing makes a view, and a write through a view changes the original. The original `kmeans` uses `X[g.choice(...)]`, an index list, which always makes a copy, so the `.copy()` there is for safety and not for need. The exercise shows what the safety protects against if a later edit changes the index to a slice.

3. [judge]

Position. Keep best-of-10 until you measure k-means++ on your own data. Then decide with the failure rate and the cost.

What a good answer weighs. On the blobs of chapter 8, 6 of 30 single starts fail, and 10 starts remove the failures. The cost is a factor of 10 in time. A smarter initialization spreads the starting centroids and reduces failures. It also costs a pass over the data. Whether it removes the need for restarts depends on the data and on `k`. This book did not measure it.

At 900 points a run takes under a millisecond, so ten starts is cheap. At 300,000 points a start takes 0.3 seconds. Three seconds for ten starts is small for a batch job and large for a request.

Strongest objection. Restarts treat a property of the initialization as a statistical problem. A better method removes it at the source. This is correct in principle. It is an argument to implement the better initialization and measure it. It is not a reason to remove the restarts before the measurement.

What overturns the position. Measure the better initialization on the real data. Suppose it fails in fewer than 1 run in 100 with a single start. Suppose also that its cost is below that of two plain starts. Then one start is enough.